

Unidad 1

Introducción a la ingeniería de software

Software. Qué es?

- Conjunto de programas en conjunto con toda la documentación requerida para definir, desarrollar y mantener los programas ejecutables que se entregan al cliente.
- Información o conocimiento que se presenta en distintos niveles de abstracción, desde los requerimientos del sistema (abstracto), hasta el código que ejecuta el mismo (detallado).

Tipos básicos de software

SYSTEM SOFTWARE	O software de sistemas, este tipo de software se encarga de controlar y gestionar los recursos del hardware de una computadora, así como de proporcionar una interfaz para que los usuarios interactúen
UTILITARIOS	Se refiere a un subconjunto de software de sistema que proporciona utilidades o herramientas adicionales para mejorar la funcionalidad del sistema operativo y el rendimiento de la computadora. Su función principal es mejorar y optimizar el rendimiento del sistema operativo y del hardware de la computadora.
SOFTWARE DE APLICACIÓN	Este software se utiliza para realizar tareas específicas en una computadora, como procesar texto, crear presentaciones o navegar por internet. Se enfoca en satisfacer las necesidades de los usuarios.

Software ≠ Manufactura

- El software es menos predecible, no como los productos tangibles en una línea de producción
- Casi ningún producto de software es igual a otro, ya que de alguna manera los requerimientos o necesidades de los usuarios que van a hacer uso de ese software van a ser distintas.
- No todas las fallas en software son errores. El tratamiento de errores en software es totalmente distinto en software que en la producción.
- El software no se gasta. Cuando hablamos de productos, normalmente, hablamos de tiempo, que al pasar del tiempo el producto se desgasta, esto no sucede con el software. Si bien el mismo debe ir adaptándose a los cambios y las necesidades del usuario/organización, específicamente no tiene la cualidad de desgastarse.
- El software (obviamente) no está gobernado por las leyes de la física.

Problemas al desarrollar/construir software

- Mas tiempos y costos que los: Este es el problema más repetido.
- Que la versión final del producto no satisfaga a las necesidades del cliente.
- Problema en la escalabilidad y/o adaptabilidad del software: Agregar una funcionalidad en otra versión es casi una misión imposible.
- Mala documentación: desactualizada y que no acompaña al software y por ende trae inconsistencias en la gestión o mantenimiento del mismo.
- Mala calidad del software: Muchas veces el software de calidad se cree relacionado con la cantidad de testing que realizo sobre él, pero esto no es así.

Ingeniería de software

- Es una disciplina que se encarga de todos los aspectos de la producción del software, desde la primera etapa de especificación del sistema hasta el mantenimiento del sistema después de que se pone en operación.

- Esta ingeniería es importante ya que:
 - Con mayor frecuencia se requiere producir de manera rápida y económica sistemas confiables.
 - Resulta más barato a largo plazo utilizar métodos y técnicas de ingeniería de software para los sistemas de software. La mayoría de los costos consisten en cambiar el software después de que se pone en operación.
- Además, la ingeniería de software se apoya en varias realidades, entre ellas:
 - En primer lugar, se debe hacer un esfuerzo para entender el problema antes de desarrollar una aplicación de software. Es decir, se debe entender el problema y las necesidades del cliente antes de proponer una solución.
 - El diseño es una actividad crucial en el desarrollo de un software.
 - Realizar un correcto diseño del software, trae consigo dos claros beneficios, calidad de software y facilidad de mantenimiento.
- La ingeniería de software nace tras la crisis del software, en la cual existía (y existe) un conjunto de dificultades en la planificación, estimación de costos, productividad y calidad de software, debido, principalmente, a la baja eficacia que presentaban algunas empresas a la hora de producir software.

Crisis del software

Hace referencia a un conjunto de hechos relativos al software, que planteó Bauer en la conferencia de la OTAN en 1968, recalcando la dificultad para producir software libre de defectos, fácilmente comprensible y que sean verificables. Sin embargo, este término fue utilizado anteriormente por Dijkstra en "El Humilde Programador".

Causas:

- La evolución de la tecnología del hardware mediante los circuitos integrados permitió la posibilidad del desarrollo de sistemas más grandes y complejos. El salto en el hardware no fue acompañado por un salto en el desarrollo del software.
- En combinación con lo anterior, la demanda creciente de estos sistemas complejos, la subestimación de la complejidad que supone el desarrollo de software, los cambios a los que tienen que ser sometidos los productos para ser adaptados a las necesidades del cliente y la falta de una disciplina que intervenga en todos los aspectos de la producción del software fueron las causas de esta crisis.

Actualidad

Muchos de los sistemas desarrollados fracasan debido a fallas en el software, que principalmente son consecuencia de dos factores:

- Demandas crecientes: a medida que la tecnología y las técnicas de desarrollo evolucionan, la demanda de sistemas más grandes y complejos aumenta. Los usuarios necesitan que se construyan y distribuyan de manera rápida. Y los métodos existentes (tradicionales) de ingeniería no permiten lograr esta "espontaneidad", por lo que se deben desarrollar y poner a prueba nuevas técnicas y métodos de gestión y desarrollo.
- Bajas expectativas: muchas de las compañías actuales de desarrollo de software no utilizan métodos de ingeniería de software en su trabajo diario. Por lo que el software con frecuencia es más costoso y menos confiable de lo que debiera. La consecuencia, recae en la conformidad de las empresas y de los usuarios ante un software. Las empresas se conforman con que el software solo "funcione", sin

tener en cuenta el valor agregado que el software le debe aportar al negocio del usuario.

Disciplinas que conforman la ingeniería de software

- Disciplinas técnicas: actividades que aportan al desarrollo de software como producto. Como: Toma de requerimientos, Análisis de requerimiento, Diseño de software, Implementación de software, Prueba de software, Despliegue de un software, Documentación de software, Capacitaciones a usuarios
- Disciplinas de gestión: hacen referencia a actividades de planificación, monitoreo y control del proyecto de desarrollo de software. Se utilizan, por ejemplo, metodologías LEAN Agile de gestión de proyectos.
- Disciplinas de soporte: son disciplinas transversales a los procesos de software, que permiten verificar la integridad y calidad de un producto de software. Se incluyen: Gestión de Configuración del Software (SCM), Toma de métricas, Aseguramiento de calidad (QA - Quality Assurance)

Proceso de software

- Conjunto estructurado de actividades que, a raíz de un conjunto de entradas, producen una salida.
- Las actividades del proceso de desarrollo son la especificación de requerimientos, el análisis, el diseño, la implementación y la prueba.
- El orden y la relación entre las diferentes actividades está determinada por la elección del ciclo de vida.
- IEEE: Un proceso es una secuencia de pasos ejecutados para un propósito dado.
- CMM: Un proceso de software es un conjunto de actividades, métodos, prácticas y transformaciones que la gente usa para desarrollar o mantener software y sus productos asociados.
- Se tienen tres factores determinantes del proceso:
 - Procedimientos y Métodos: Definición de cómo vamos a llevar a cabo las actividades de desarrollo. Definir las actividades que se harán y cuáles no.
 - Personas motivadas, capacitadas y con habilidades: Es factor primordial para obtener la calidad del producto del proceso, ya que éste se determina del esfuerzo de las personas. Sin ellas, no se puede lograr construir ningún producto.
 - Herramientas y Equipos: Materiales necesarios para llevar a cabo el proceso que nos permiten que las entradas se transformen en salidas. Se recomienda automatizar la mayor cantidad de actividades posibles.

Al proceso de desarrollo de software lo define la organización en función de los objetivos y de las características del software.

Proceso definido:

- Se basan en una concepción basada en el modelo industrial (inspirados en líneas de producción), en la cual tengo un conjunto de inputs que van a ser utilizados en un conjunto dado de actividades, que van a producir el mismo resultado o salida siempre que los inputs sean los mismos.
- Esto es difícilmente aplicable al software, ya que no es tan definible el software. No podemos simplificar el software a una entrada de inputs que siendo utilizado para determinadas aplicaciones produce el mismo output, es decir que el output es predecible.

- Asume que podemos repetir el proceso indefinidamente y obtener los mismos resultados.
- La administración y el control provienen de la predictibilidad del proceso definido. Muchas corrientes y procesos confían en estas premisas para llevar a cabo un proyecto de software.

Procesos Empíricos

- Viene del empirismo, es aquella corriente en la cual centramos el foco en la experiencia. (Más adelante sus pilares). No solo la experiencia del usuario sino también la experiencia en el negocio, la experiencia técnica, en cómo voy a manipular las herramientas y aprender de los errores para poder aplicarlo.
- Nos basamos en capitalizar la experiencia de los actores involucrados en la producción para maximizar la calidad de lo que estamos produciendo.
- Vamos a tomarlos en contraste con los procesos definidos. Confiamos en que las variables no pueden ser definidas o controladas en su totalidad, estamos dentro de un ambiente complejo donde quizás no todo lo que está en juego está bajo nuestro control y esto nos implica la necesidad de adaptación para poder garantizar la producción de un software de calidad.
- Se basan en la experiencia y el aprendizaje, y la principal diferencia con los procesos definidos, es la forma en la cual se puede mejorar el proceso. Mientras que en el proceso definido el punto de mejora es el central (las actividades/etapas intermedias, ya que mis entradas y salidas son fijas), el proceso empírico no tiene un punto de mejora tan tangible, voy a tener que ir trabajando sobre distintos aspectos, siempre centrándome en las personas que son la clave del empirismo, son quienes capitalizan la experiencia.
- Esta capitalización de la que hablamos se hace en un ciclo de retroalimentación que permite la adaptación y mejora, aprendemos de la experiencia.

Ciclos de vida y su influencia en la administración de proyectos de software

- El ciclo de vida es una abstracción.
- Me va a definir cuáles son las etapas (fases) y el orden de cada una de ellas.
- No me dice que tengo que hacer, ni quien lo va a hacer, ni las herramientas ni los procedimientos sino solo las fases y en qué orden se van a ejecutar.
- Son la serie de pasos a través de los cuales un producto o un proceso progresa.

Relación entre el ciclo de vida del proyecto y del producto

Proyecto	Producto
• El cv de un proyecto de software es una representación de un proceso. • Grafica una descripción del proceso desde una perspectiva particular. • nos define las fases del proceso y el orden en el cual se llevan a cabo las mismas. • Podemos decir que el cv de un proyecto empieza y termina.	• El cv de un proyecto termina cuando genera un producto, cuyo cv no va a terminar, sino que va a mantenerse mientras el objeto exista. • El producto va a necesitar mantenimiento una vez que “termine” de desarrollarse, y se va a mantener bajo este proceso de mantenimiento hasta que se deseche, ahí podemos decir que el cv del producto llega a su fin.

Es posible que un producto tenga varios proyectos en su ciclo de vida, debido a, constantes cambios y/o actualizaciones que se vayan realizando.

Clasificación de los ciclos de vida:

- **Modelo Secuencial**
 - Dispone de las actividades de forma lineal. El proyecto progresa a través de una secuencia ordenada de pasos (fases) y una actividad no puede iniciar sin que la precedente haya sido finalizada (sin retorno generalmente).
 - El avance entre las fases se da tras una revisión al final de cada una de estas para determinar si el software está listo para avanzar.
 - Se suele utilizar en aquellos sistemas donde los requerimientos se comprenden bien y son exhaustivos ya que éstos se congelan al finalizar la etapa de toma de requerimientos.
 - Generalmente, este tipo de modelos está dirigido por documentos, es decir, el trabajo principal del producto es la documentación del software entre las distintas fases.
 - **Escenarios de proyectos donde elegirlo: Modelo en cascada.**
- **Modelo iterativo/incremental**
 - Aplica sucesivas iteraciones en forma escalonada a medida que avanza el calendario de actividades.
 - Cada iteración produce un incremento de software funcional potencialmente entregable.
 - Usualmente los primeros incrementos incluyen las funciones básicas/críticas que más requiere el cliente.
 - Este enfoque vincula las actividades de especificación, desarrollo y validación.
 - **Se utiliza en metodologías ágiles, pero también en otros como PUD.**
 - **En ágil solo se usa modelo iterativo/incremental, pero el iterativo/incremental no es solo de ágil.**
- **Modelo Recursivo (está en desuso)**
 - Se utiliza para casos particulares como proyectos de alto riesgo.
 - Dentro de este tipo de ciclos tenemos el ciclo de vida de espiral, que se basa en la investigación de riesgos y el ciclo de vida en B que permite hacer una vuelta en particular de cada una de las etapas e ir avanzando en los incrementos.
 - Se basa en tomar una característica específica y en una iteración me centro en esa funcionalidad, en la siguiente en otra, y así.

Administración de proyectos dependiendo en ciclo de vida

- La elección de un ciclo de vida afecta de forma directa a la administración que se debe realizar sobre el proyecto, ya que cada uno de los modelos de proceso poseen características y enfoques distintos.
- Además, durante el desarrollo del proyecto, la gestión de este se realiza en consecuencia de esta elección, ya que no es lo mismo gestionar un proyecto que se desarrolla de manera iterativa, que uno que se desarrolla de manera secuencial o en cascada.

Unidad 2

Software Configuration Management (SCM)

- Los sistemas de software siempre cambian durante su desarrollo y uso.
- Conforme se hacen cambios al software, se crean nuevas versiones del sistema.
- Los sistemas pueden considerarse como un conjunto de versiones donde cada una de ellas debe mantenerse y gestionarse. Esto es necesario para no perder la trazabilidad de los cambios que se incorporan a cada versión.

Definición de Gestión de Configuración

- Es una disciplina de soporte, transversal a todo el proyecto con aplicación en diferentes disciplinas, que surge para enfrentar la crisis de problemas de software, aplicando dirección y monitoreo (administrativo y técnico) a:
 - Identificar y Documentar características técnicas y funcionales de ítems de configuración.
 - Controlar los cambios de esas características identificadas y documentadas;
 - Registrar y reportar estos cambios;
 - Verificar correspondencia con los requerimientos (trazabilidad).
- Su propósito es resolver problemas de diferente índole, a través del establecimiento y el mantenimiento de la integridad del producto de software a lo largo de todo su ciclo de vida.
 - Ej de problemas: Pérdida de componentes; Pérdida de cambios (la versión del componente no es la última); Regresión de fallas; Doble mantenimiento; Superposición de cambios; Cambios no validados.
- Su propósito es establecer y mantener la integridad de los productos de software a lo largo de su ciclo de vida.
 - Esto implica identificar la configuración en un momento dado, controlar sistemáticamente sus cambios y mantener su integridad y origen.
- La integridad es el medio por el cual podemos garantizar que el producto a entregar tiene la calidad correspondiente. Y nos garantiza un nivel mínimo de confiabilidad.
 - La idea de la integridad del producto es hacer explícita las características o expectativas que tiene el cliente sobre el producto de software.
 - se mantiene la integridad de un producto de software cuando:
 - Satisface las necesidades de usuario.
 - Permite la rastreabilidad durante todo su ciclo de vida.
 - Satisface criterios de performance y RNF.
 - Cumple con expectativas de costo.

¿Cuál es el problema que se está tratando de solucionar?

- El software es muy fácil de modificar, en el sentido de que uno puede entrar al repositorio y con un solo click eliminar meses de trabajo.
- Lo que buscamos con la gestión de configuración del software es tratar de el producto de software sea íntegro y si fuera el caso de que se van a realizar cambios, podamos tener un control de qué cosas cambiaron en un determinado momento.
- Es por esta razón que se asocia el concepto de software con el concepto de versión, ya que cada ítem de configuración debe tener su versión.

Conceptos Clave para la Gestión de Configuración de Software

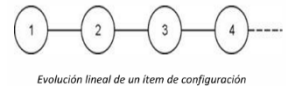
Ítem de Configuración de Software (SCI)

- Son aquellos artefactos que forman parte del producto o proyecto, y pueden ser almacenados en un repositorio, sin importar su extensión/tipo. Pueden sufrir cambios, y se desea conocer su estado y evolución a lo largo del ciclo de vida (ya sea del producto o proyecto, dependiendo del artefacto).
- Es cualquier aspecto asociado al producto de software (requerimientos, diseño, código, datos de prueba, documentos, etc.) que es necesario mantener.

- Son todos aquellos elementos que componen toda la información producida como parte del proceso de ingeniería de software, como ser programas de computadora (código fuente y ejecutables), documentos que describen los programas (documentos técnicos o de usuario), datos (de programa o externos).
- Los ítems, dependiendo de su naturaleza, pueden durar lo que dure el ciclo de vida del proyecto, producto o sprint.
- Algunos ejemplos: prototipo de interfaz, manual de usuario, ERS, arquitectura del software, casos de prueba, código fuente, íconos, etc.

Versión

- Instancia de un ítem de configuración que difiere de otras instancias de este ítem.
 - Las versiones se identifican unívocamente.
 - Controlar una versión refiere a la evolución de un único ítem de configuración.
- Una versión se define, desde el punto de vista de la evolución, como la forma particular de un artefacto en un instante o contexto dado.
- "La gestión de configuración permite a un usuario especificar configuraciones alternativas del sistema de software mediante la selección de las versiones adecuadas"; esto se puede gestionar asociando atributos a cada versión (que pueden ser datos sencillos como un nro. de versión asociado a cada objeto. Cada versión de software es una colección de elementos de configuración (ECS), como ser código fuente, documentos, datos).
- La evolución puede representarse gráficamente en forma de grafo.



Variante

- Versión de un ítem de configuración (o de la configuración) que evoluciona por separado. Las variantes representan configuraciones alternativas.
- Las variantes de un ítem de configuración permiten satisfacer requerimientos en conflicto.
 - Por ejemplo la compatibilidad del producto con diferente hardware o sistemas operativos.
- Si la diferencia que existe entre las instancias de un mismo ítem de configuración es muy pequeña, no se denomina versión, sino que se la conoce como variante.
- Está ligado a trabajar con Branches.

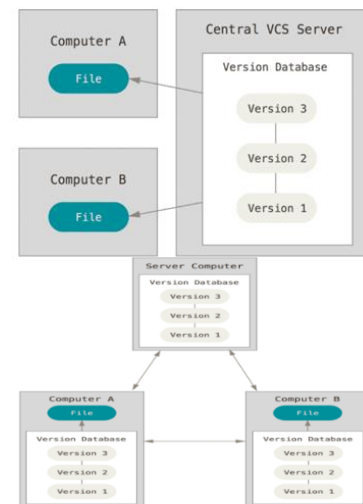
La Configuración del Software

- Es la sumatoria de todos los ítems de configuración que tiene en un momento determinado, equivale a una instantánea o una foto de todos los ítems de configuración con su versión en un momento del tiempo.
- Conjunto de todos los componentes fuentes que son compilados, sus documentos y la información de la estructura que definen una versión del producto a entregar.

Repositorio

- Es un contenedor de ítems de configuración.
- Se encarga de mantener la historia de cada ítem con sus atributos y relaciones, además es usado para hacer evaluaciones de impacto de los cambios propuestos.
- Puede ser una o varias bases de datos.

- Tiene una estructura para mantener el orden y la integridad.
- Tenemos 2 tipos de repositorios:
 - Repositorios Centralizados: está caracterizado porque un servidor contiene todos los archivos con sus versiones. Los administradores tienen un mayor control sobre el repositorio. Tiene como desventaja que si falla el servidor, todo lo positivo se cae.
 - Repositorios Descentralizados: está caracterizado porque cada cliente tiene una copia exactamente igual del repositorio completo. Tiene como ventaja que se resuelve el problema de los servidores centralizados, debido a que si el servidor falla sólo es cuestión de “copiar y pegar”, además posibilita otros workflows no disponibles en el modelo centralizado. La desventaja que podemos mencionar es que es más complicado llevar un control sobre el mismo.



Rama (Branch)

- Existe una rama principal (trunk, master)
- Es un conjunto de ítems de configuración con sus correspondientes versiones, que permiten bifurcar el desarrollo de un software, por varios motivos:
 - Experimentación;
 - Resolución de errores en el desarrollo;
 - Desarrollar un mismo software para diferentes plataformas.
- Pueden ser descartadas o integradas.
- Integración de ramas → MERGE
 - Lleva los cambios a la rama principal
 - Pueden surgir conflictos (resolvemos con diff)
 - Todas las ramas deberían eventualmente integrarse a la principal o ser descartadas

Líneas base y su administración

- Conjunto de ítems de configuración que han sido construidos y revisados formalmente, de manera que pueden ser tomados como referencia para demostrar que se ha alcanzado cierto nivel de madurez en ellos, y que sirve como base para desarrollos posteriores.
- Este conjunto de ítems debe tener una referencia única, y esto se hace a través de “tags”.
- Se acuerdan parámetros para determinar qué se considera o qué debe cumplir los ítems de configuración para considerarse como línea base.
- Si se desea cambiar una línea base, existe un protocolo formal de control de cambios, dirigido por el Comité de Control de Cambios, que permite definir el procedimiento a seguir para manejar peticiones de cambios, y en caso de aceptarse, acordar nuevamente los parámetros y comunicar los cambios a todo el equipo, para que tomen la nueva línea base como modelo a seguir.
- Sirven para:
 - Fundamentalmente es para tener un punto de referencia, pero también sirve para hacer Rollback o saber qué se pone en producción.
 - Nos permite saber cuál era la última situación estable en un momento de tiempo y cómo se fue evolucionando.

- Permiten ir atrás en el tiempo y reproducir el entorno de desarrollo en un momento dado del proyecto.
- Tipos de línea base:
 - Operacionales: contiene una versión de producto cuyo código es ejecutable, han pasado por un control de calidad definido previamente. La primera línea base operacional corresponde con la primera Release.
 - Es la línea base de productos que han pasado por un control de calidad definido previamente.
 - De especificación o documentación: son las primeras línea base, dado que no cuentan con código. Podría contener el documento de especificación de requerimiento.
 - Son de requerimientos, diseño.

Planificación de la gestión de configuración de software

- Este plan se debe confeccionar al inicio de un proyecto, y existen diferentes estándares donde se expresa cómo planificar: Reglas de nombrado de los ítems de configuración; Herramientas para utilizar en SCM; Roles e integrantes del Comité de Control de Cambios; Procedimientos formales de cambios; Procesos de auditoría; Estructura del repositorio; SCM para software externo (opcional); Cómo se hará el control de cambios; Registros que deben mantenerse; Tipos de documentos.
- Consideraciones: Debe hacerse tempranamente; Se deben definir los documentos que se administrarán; todos los productos del proceso deben administrarse.

Actividades relacionadas a la gestión de configuración

- Estas actividades/elementos son las cosas que contienen las secciones de un plan de gestión de configuración.
- Todas las actividades se realizan en ambas metodologías, menos las auditorías, ya que no es compatible con una metodología ágil pura.

Identificación de Ítems de Configuración

- Implica una **identificación unívoca para cada ítem**, donde en el equipo se definirán **políticas y reglas de nombrado y versionado** para todos ellos.
- Se debe **definir la estructura del repositorio**, y **la ubicación de los ítems de configuración dentro de esa estructura**.
- Implica una definición cuidadosa de los componentes de la línea base.
- Esta identificación y documentación proveen un camino que une todas las etapas del ciclo de vida del software, lo que permite a los desarrolladores controlar y velar por la integridad del producto, como así también a los clientes evaluar esa integridad.
- También se debe tener en cuenta al momento de identificar los ítems, la duración de su integridad, ya que difieren en función del tiempo que es necesario mantenerlos.
- Se clasifican en:
 - Ítems de producto: tienen el ciclo de vida más largo, y se mantienen mientras el producto exista. Ejemplo: documento de arquitectura, casos de uso, código, manual de usuario y de parametrización, casos de prueba, una ERS, los casos de prueba, la base de datos.
 - Ítems de proyecto: el plan de proyecto, el listado de defectos encontrados, tienen un ciclo de vida de proyecto. Un plan de iteración, un Burn-Down

Chart, se mantiene durante una iteración. La duración impacta también en el esquema de nombrado, plan de proyecto. Ejemplo: Los ítems de configuración a nivel de proyecto, el plan de proyecto y el cronograma.

- Ítems de iteración: Conocer el ciclo de vida de un ítem permite establecer la nomenclatura de este. Ejemplo: planes de iteración, cronogramas de iteración, reporte de defectos.
- Luego de identificar los ítems se debe armar la estructura del repositorio con nombres representativos.
 - Luego se vinculan los ítems de configuración con el lugar físico del repositorio donde se van a almacenar.

Control de cambios

- Una vez definida la línea base, no es posible cambiarla sin antes pasar por un proceso formal de control de cambios.
- El control se hace sobre los ítems de configuración que pertenecen a la línea base, ya que todos los trabajadores del software tienen a dichos ítems como referencia.
- Este proceso es llevado a cabo por el comité de control de cambios, el cual se reúne para autorizar la creación y cambios sobre la línea base.
 - Forman parte de dicho comité los interesados en evaluar y en enterarse del cambio, decidiendo si lo aceptan o no.
- Etapas en caso de recibir una propuesta de cambios sobre una línea base:
 - Se recibe una propuesta de cambio sobre una línea base determinada, no sobre un ítem.
 - El comité de control de cambios realiza un análisis de impacto del cambio para evaluar el esfuerzo técnico, el impacto en la gestión de los recursos, los efectos secundarios y el impacto global sobre la funcionalidad y la arquitectura del producto.
 - En caso de que se autorice la propuesta de cambio, se genera una orden de cambio que define lo que se va a realizar, las restricciones a tener en cuenta y los criterios para revisar y auditar.
 - Luego de realizado el cambio, el comité vuelve a intervenir para aprobar la modificación de la línea base y marcarla como línea base nuevamente, es decir que realiza una revisión de partes.
 - Finalmente se notifica a los interesados los cambios realizados sobre la línea base.

Auditorías de configuración

- En metodologías ágiles, esta es la única actividad de la disciplina SCM que no se soporta por la filosofía.
- Son controles autorizados a realizar por el equipo de desarrollo en un momento determinado, donde un auditor externo al equipo (independiente y objetivo) analiza las líneas base, las cuales permiten “congelar” y analizar en un momento determinado cuál es el estado del software, y si se están cumpliendo todas las pautas que plantea esta disciplina de SCM.
- La auditoría de configuración se hace mientras el producto se está construyendo y su objetivo es que el producto tenga calidad e integridad. Se entiende que lo más barato es prevenir.
- complementa a la revisión técnica y consiste en revisar si se están realizando las tareas tales como se planificaron y especificaron en el plan de gestión de configuración.

- Las auditorías se realizan sobre una línea base.
- Trazabilidad o rastreabilidad de requerimientos:
 - Establecer una correspondencia de relación entre ítems de configuración de diferentes niveles de abstracción en el proceso de desarrollo de software.
 - La trazabilidad permite realizar un análisis del impacto sobre el sistema a partir de una solicitud de cambio en el requerimiento.
- Procesos a los que sirve:
 - Validación: se encarga de asegurar que cada ítem de configuración resuelva el problema apropiado, es decir, lo que el cliente necesita.
 - Verificación: implica asegurar que un producto cumple con los objetivos definidos en la documentación de líneas base. Todas las funciones son llevadas a cabo con éxito y los test cases tengan status “ok” o bien consten como “problemas reportados” en la nota de reléase.
- Tipos de auditorías:
 - Auditoría física o de contingencia:
 - se realiza primero.
 - Se validan los ítems de configuración de la línea base contra el plan de gestión de configuración, asegurando que la documentación que se entregará es consistente con el código desarrollado. Si esto no pasa, la auditoría funcional no se realiza.
 - Controla que el repositorio esté alojado en el lugar donde se dijo que iba a estar, que los IC respeten el esquema de nombrado, que estén guardados donde se dijo que iban a estar guardados.
 - Lo que se audita es una línea base.
 - Hace **verificación**.
 - 2. Auditoría funcional de configuración:
 - Audita la trazabilidad de requerimientos, controlando que la funcionalidad y performance real de cada ítem de configuración sean consistentes con la especificación de requerimientos.
 - **El propósito final es validar que el código implementa únicamente y todos los requerimientos de la ERS.**
 - Evaluación independiente de los productos de software, controlando que la funcionalidad y performance reales de cada ítem de configuración sean consistentes con la especificación de requerimientos.
 - Antes de comenzar, pregunta si está disponible el reporte de la auditoría de configuración física. Ya que, si la auditoría física no sale bien, la auditoría funcional no se hace.
 - Hace **validación**.

Informe de estado

- Provee un mecanismo para mantener un registro de cómo evoluciona el sistema, y dónde está ubicado el software, comparado con lo que está publicado en la línea base.
- Permite mantener a todo el equipo informado sobre la última línea base, y que se trabaje en base a su última versión, y no sobre una versión obsoleta.
- Estos reportes incluyen todos los cambios que han sido realizados a las líneas base durante el ciclo de vida del software.
- El objetivo principal es asegurarse que la información de los cambios llegue a todos los involucrados.

- El reporte más conocido es el de inventario, el cual provee una copia del contenido del repositorio con la estructura de directorios.

Gestión de configuración en ambientes ágiles

- En las metodologías ágiles la gestión de configuración cambia el enfoque, ya que la misma es de utilidad para los miembros del equipo de desarrollo y no viceversa.
- Decimos que la gestión de configuración posibilita el seguimiento y la coordinación del desarrollo en lugar de controlar a los desarrolladores.
- Los equipos ágiles son autoorganizados, por lo que las Auditorías de configuración no son una actividad propia de la gestión de configuración en los ambientes ágiles.
- La diferencia es que el manifiesto ágil se enfoca en los proyectos, mientras que la gestión de configuración de software se enfoca en el producto. Es decir, trasciende el proyecto.
- Mientras el manifiesto ágil apunta a cómo trabajar en el contexto del proyecto de desarrollo y la gestión de configuración es una práctica específica del producto que garantiza la calidad del producto.
- Características:
 - Dada la naturaleza de los requerimientos ágiles, la SCM responde a los cambios en lugar de evitarlos.
 - La automatización debe ser aplicada donde sea posible. Esto colabora con la entrega frecuente y rápida de software (integración continua).
 - Provee retroalimentación continua sobre calidad, estabilidad e integridad.
 - Las actividades de SCM se encuentran embebidas en las demás tareas para alcanzar el objetivo del sprint.
 - Es responsabilidad de todo el equipo, por lo tanto, requiere capacitación.
 - Adopción de las prácticas continuas.

Continuous Integration

- Es una práctica de desarrollo que promueve que los desarrolladores adopten la costumbre de integrar su código a un repositorio compartido varias veces al día.
- Cada integración de código es luego verificada por una serie de pruebas automatizadas, permitiéndole al equipo detectar problemas de manera temprana.
- Ya que al integrar el código al repositorio de manera frecuente resulta más fácil detectar y corregir los errores.
- La integración continua es asegurar que el software pueda ser desplegado en cualquier momento, es decir, que el código compile y que sea de calidad.
- Cada desarrollador en su entorno de trabajo realiza pruebas unitarias (en la medida de lo posible, automatizadas) desarrollando con algo que se llama TDD (desarrollo conducido por pruebas) y cuando terminó ese componente de código y lo probó y sabe que funciona, lo sube a un repositorio de integración.

Continuous Delivery

- Es una disciplina de desarrollo de software en la que el software se construye de tal manera que puede ser liberado en producción en cualquier momento.
- Suma la automatización de las pruebas de aceptación y entonces el producto ya está listo para desplegarlo a producción.
- No implica necesariamente que se libere cada vez que hay un cambio, sólo ocurre si el responsable de negocio, el Product Owner de turno, decide pasar a producción. Es decir que hay un componente «humano» a la hora de tomar la decisión. Pero, en cualquier caso, la versión está lista de inmediato.

- Esto implica, entre otros, que se prioriza que la versión esté en un estado en el que pueda ser puesta en producción.
- La integración continua es prerequisite de la entrega continua. Con esto se refiere a que los artefactos producidos en el servidor de integración continua son puestos en producción con solo un click. Hay que asegurarse de tener un despliegue automatizado, para que cada puesta en producción no lleve demasiado tiempo, lo que hará que las puestas puedan llegar a hacerse más seguidas, generando que lo que se despliegue no sean demasiados cambios y el riesgo

Manifiesto Ágil

- Es un documento redactado en 2001 por expertos de la Ingeniería en Software, que promueve la filosofía del desarrollo ágil del software.
- Se trata de un compromiso que hacen todas las personas involucradas en un proyecto para trabajar de una determinada manera independientemente de las prácticas que realice cada uno.
- El objetivo del Movimiento Ágil era lograr un equilibrio entre seguir procesos rigurosos y formales en el desarrollo de software y trabajar de manera eficiente y efectiva para entregar un producto de calidad.
- También se suele llamar AGILE. No es una metodología ni un proceso, es una ideología que cuenta con un conjunto definido de principios que guían el desarrollo del producto. Busca encontrar un equilibrio entre procesos definidos y nada de procesos.
- El manifiesto ágil se sustenta en los **procesos empíricos** que tienen como base la experiencia, y la misma sale del propio equipo, por ello es importante tener ciclos de realimentación cortos.
 - Esto implica comenzar con una idea o requisito, construir un producto mínimo viable (MVP) y mostrarlo al cliente para obtener retroalimentación.
 - A partir de esa retroalimentación, se realizan mejoras y correcciones continuas durante todo el proceso de desarrollo.
 - Este enfoque es compatible únicamente con ciclos de vida iterativos y se basa en la idea de que la incertidumbre y los cambios son inevitables en el desarrollo de software.

El empirismo tiene 3 pilares:

- **TRANSPARENCIA:** Se refiere a la comunicación abierta y honesta de la información relevante a todas las partes interesadas. La transparencia promueve la confianza y la colaboración entre los miembros del equipo y las partes interesadas, lo que a su vez conduce a una toma de decisiones más informada y eficaz.
- **ADAPTACIÓN:** Se refiere a la capacidad de adaptarse y ajustar el trabajo y el proceso para hacer frente a las desviaciones y problemas detectados durante la inspección. La adaptación permite una respuesta rápida y efectiva a los cambios y permite mantener la entrega de valor al cliente.
- **INSPECCIÓN:** Se refiere a la revisión regular y sistemática del progreso del trabajo y los productos resultantes para detectar problemas y desviaciones del plan. La inspección permite una evaluación temprana y continua del progreso y ayuda a tomar decisiones informadas sobre los próximos pasos.

Valores ágiles

- Individuos e interacciones por sobre procesos y herramientas

- Se enfatizan los vínculos, la comunicación efectiva y la colaboración entre los miembros del equipo y las partes interesadas, por sobre la adopción de la herramienta que tenemos que usar y el proceso a aplicar.
- El desarrollo de software es una actividad humano-intensiva, y los procesos ágiles intentan capitalizar las fortalezas de cada individuo.
- Las dos cosas son importantes sólo que los individuos e interacciones son más valorados.
- Software funcionando por sobre documentación extensiva
 - Este valor enfatiza la importancia de proporcionar software funcional y de alta calidad en lugar de enfocarse en documentar cada detalle del proceso de desarrollo.
 - A medida que el software crezca en cuanto a funcionalidades, tras cada iteración puede ir mostrándose a los usuarios finales y obtener una retroalimentación de ellos, para que el equipo de desarrollo se asegure de estar trabajando siempre en aquellas funcionalidades de mayor valor para el cliente y aquellas que satisfagan sus expectativas.
 - Esto no quiere decir que NO se debe generar documentación.
 Las **decisiones tomadas con respecto al producto de software deben quedar documentadas** y van a trascender más allá de la iteración en la cual se generó el incremento al mismo. Lo que se debe decidir es: qué aspectos del producto van a quedar documentados, teniendo en cuenta que el conocimiento del producto debe ser transparente y de toda la organización no de un individuo. **Con respecto al proyecto**, lo mismo, **vamos a definir qué vamos a documentar del proyecto**. Otro aspecto a tener en cuenta es la permanencia, no toda la información que se genera debe tener permanencia y disponibilidad infinita.
Se debe generar la documentación cuándo sea necesario. No plantea “no generar documentación”.
- Colaboración con el cliente por sobre negociación contractual (o de contratos)
 - Este valor enfatiza la importancia de trabajar en colaboración con el cliente y comprender sus necesidades y requisitos en lugar de simplemente negociar contratos y acuerdo formales.
 - Está muy relacionado con la triple restricción.
 - Los problemas en las negociaciones contractuales se dan cuando el cliente firma un contrato con el equipo definiendo un alcance/costo/tiempo determinado y luego quiere realizar cambios en el mismo.
 - No se trata de tener a un cliente lejano que no sabe lo que sucede durante el desarrollo, si no tener a uno que esté comprometido con cada entrega, sobre todo al momento de probarlo y dar el feedback.
 - A veces las negociaciones contractuales imponen ciertas restricciones que impactan de manera negativa en las partes, lo cual se traduce en un mayor distanciamiento en la relación equipo de desarrollo-cliente.
 - Es fundamental que el cliente esté dispuesto a participar y sumarse a este enfoque, ya que, de lo contrario, la metodología Ágil no se puede aplicar.
 - Por lo tanto, no sólo es importante formar a los equipos que trabajarán con enfoque Ágil, sino también al cliente (quien en Scrum es el Product Owner).
- Responder a cambios por sobre seguir un plan
 - Este valor enfatiza la importancia de ser flexible y adaptativo en la respuesta a cambios y desviaciones en el proceso de desarrollo en lugar de seguir un plan rígido y predefinido que puede no ser el adecuado para el contexto actual.

- Plantea que debemos construir en conjunto con el cliente, no definir tanto, empezar a trabajar con una idea del producto y después enfocarnos más a medida que avanzamos para darle la posibilidad al cliente de cambiar de opinión.
- Es imposible que los usuarios sepan con detalle y certeza todas las funcionalidades que necesitan, y a su vez, con el paso del tiempo surgirán nuevas necesidades, las cuales en un primer momento no se habían detectado.

Principios del Manifiesto Ágil

- Nuestra mayor prioridad es satisfacer al cliente:
 - Ligado al valor 3 y 4, debido a que un equipo ágil desarrolla y entrega funcionalidades en el orden especificado por el Product Owner.
 - Es ese rol quien determina la prioridad e importancia de cada funcionalidad o Feature, de manera que en cada incremento o Release del producto, se esté retornando al cliente el mayor valor posible.
 - Esto está soportado por el hecho de trabajar con User Stories, las cuales plantean necesidades que tienen un valor significativo para los usuarios del software.
- Aceptar que los requisitos cambien, incluso en etapas tardías de desarrollo:
 - Ligado a los valores 3 y 4, donde se plantea que el equipo ágil está abierto al cambio, al no seguir un plan estructurado que se confecciona en etapas tempranas, como plantea la metodología tradicional.
 - Permite que el equipo sea flexible, y responda siempre ante las necesidades cambiantes de los clientes y/o usuarios.
- Entregar software funcional frecuentemente:
 - Esto se relaciona al valor 2, donde es imprescindible que un equipo ágil al finalizar cada iteración, haya sido capaz de transformar uno o más requerimientos en código desarrollado completamente, testeado y potencialmente desplegable.
 - Es importante destacar que ese código pueda ser desplegado debido a que se debe asegurar que cumpla con lo que el usuario/cliente necesita, de forma que se le entregue el mayor valor posible tras cada iteración, y no entregar una funcionalidad desarrollada parcialmente.
- Técnicos y no técnicos trabajando juntos durante todo el proyecto:
 - Valores 1 y 3, plantea la gran importancia de la participación del “Product Owner”, en conjunto con el equipo de desarrollo de software. Este rol, que generalmente ocupado por una persona del “lado del cliente”, debe ser alguien capaz de tomar decisiones sobre el producto, y que además sea capaz de representar el interés y necesidades comunes de todos los usuarios a los que estará destinado el desarrollo del software.
 - En un enfoque de proceso ágil, la participación del Product Owner es de suma importancia, ya que es quien guía y prioriza tras cada iteración, aquellos requerimientos que serán implementados. Además, generalmente es el encargado de escribir las User Stories,
 - Comúnmente, el P.O. no posee esa disponibilidad que plantea el enfoque ágil, convirtiéndose en el talón de Aquiles en el proceso de desarrollo del software y siendo uno de los motivos por los que se fracasa en los enfoques ágiles.
- Desarrollamos proyectos en torno a individuos motivados:

- este principio está ligado al valor 1, donde la filosofía asume que aquellos individuos que participan en un proyecto de desarrollo del software deben estar lo suficiente motivados y comprometidos con los objetivos de este.
 - Los participantes de un proyecto se deben ver entre sí como a un único equipo con objetivos comunes, donde las personas que asumen diferentes roles intentan colaborar entre sí para cumplir con esos objetivos.
- El método más eficiente de comunicar información es con conversaciones cara a cara:
 - ligado con el valor 1, este principio tiene un sustento psicológico, el cual expresa que las palabras expresadas como sonido o texto, sólo representan un bajo porcentaje de la comunicación que se desea lograr hacia otra persona, siendo que la mayor parte está ligada a gestos, expresiones faciales, movimientos corporales, retroalimentación rápida, etc.
- La mejor métrica de progreso es el software funcionando:
 - ligado con el valor 2, donde el manifiesto ágil plantea que la mejor forma de demostrar el avance en un proyecto de software es a través de la entrega de software funcional y de valor para el cliente.
 - En las metodologías tradicionales existen muchas métricas y documentaciones sobre las cuales se basan para medir el avance de un proyecto, pero poco sentido tiene tener una excelente documentación del proyecto y producto, si el producto en sí no refleja el valor que se intenta entregar al cliente.
- Los procesos ágiles promueven el desarrollo sostenible
 - Ligado con los valores 2 y 4, esto apunta a la estabilidad en equipos de trabajo a lo largo del tiempo, lo cual se traduce en una mejora continua de relaciones y formas de trabajo del equipo.
 - Mantener un equipo estable a lo largo del tiempo permite que el equipo alcance una madurez tal, en la cual es mucho más fácil realizar estimaciones sobre su participación en el proyecto.
- La atención continua a la excelencia técnica y al buen diseño mejora la agilidad:
 - Esto se relaciona al valor 2, y manifiesta que la calidad del producto no es un aspecto negociable.
 - Es de suma importancia que a medida que el software se desarrolla, se atiendan las necesidades primordiales del cliente y se responda a ellas, entregando al cliente código de calidad.
- La simplicidad es esencial:
 - Ligado a los valores 2 y 3, este principio apunta a que, como equipo de desarrollo, nos centremos en aquellas necesidades que realmente aporten un valor al cliente, y no perder el tiempo en detalles o aspectos que no se traducen en satisfacer requerimientos de los usuarios, ni hacer desarrollos de más que no estén especificados como necesidad del cliente.
- Las mejores arquitecturas, diseños y requerimientos surgen de equipos autoorganizados:
 - Ligado al valor 1, este principio expresa que los equipos de desarrollo ágiles a diferencia de organizaciones jerárquicas tradicionales, donde la dirección/gerencia o los jefes son quienes imparten órdenes, y todos deben acatarlas, bajo esta filosofía los integrantes del equipo deben ser proactivos, aportando soluciones, conceptos, herramientas, etc. que sean útiles al equipo.
 - Este principio está sustentado por el empirismo, el cual establece que el conocimiento surge dentro del equipo de trabajo.

- El equipo toma las decisiones en el momento que ellos creen adecuados.
- A intervalos regulares, el equipo evalúa su desempeño y ajusta la manera de trabajar:
 - Ligado a los valores 1 y 4, este principio se basa en los dos pilares de los procesos empíricos (inspección y adaptación), los cuales permiten a los equipos que cada determinados períodos de tiempo evalúen distintos aspectos concernientes a su desempeño, tales como variaciones de personal en el equipo, tecnologías que no han funcionado como se esperaba, cambio de ideas de usuarios, competencia laboral, riesgos, nuevas tecnologías en uso y reflexionar sobre oportunidades de mejoras en general.
 - Un equipo ágil debe inspeccionar su forma de desempeño, para adaptar aquellos aspectos que sean necesarios corregir, mejorar o incorporar al equipo, de manera que el comportamiento del equipo como un todo mejore, y de esa forma se pueda responder a las necesidades cambiantes de los clientes.

Filosofía Ágil

- El objetivo primordial es entregar software de forma frecuente en un entorno cambiante.
- Estas metodologías se utilizan en entornos con gran variabilidad de requerimientos, involucrando al cliente en el proceso de desarrollo para conseguir una rápida retroalimentación.
- Adopta el ciclo de vida iterativo-incremental, donde el concepto del empirismo subyace al proceso, el software no se desarrolla como una sola unidad, sino como una serie de incrementos y cada uno de ellos incluye una nueva funcionalidad del sistema.
- Es un compromiso entre nada de proceso y demasiado proceso”.
- Los métodos ágiles son adaptables en lugar de predictivos y están orientados a la gente en lugar del proceso.

¿Cuándo es aplicable ágil?

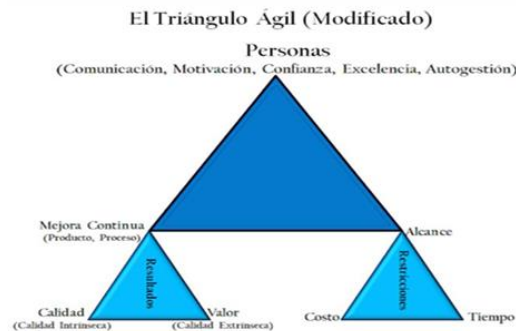
- Ágil no es una metodología o un proceso, ni tampoco un framework completamente definido, sino que es una ideología con un objetivo definido de valores y principios que guían el desarrollo del producto.
- Los frameworks ágiles son distintos marcos de trabajo para implementar los principios y valores ágiles y generalmente son orientados a la gestión de proyectos y no al desarrollo.
- Son aplicables en el contexto de lo complejo, en el cual se ubican los nuevos productos sobre los cuales el entendimiento y la certeza que se tiene es parcial.

El triángulo ágil

- El triángulo ágil apunta a que lo que realmente tiene valor que es el software funcionando para el cliente, por lo que se centra más en la calidad de producto, es decir, en la calidad intrínseca (**calidad** que nosotros le ponemos al producto para que el producto pueda satisfacer al cliente) y la calidad extrínseca (el **valor** que el producto tiene para el cliente).
- El triángulo ágil (modificado) apunta a un producto de software que pone a las personas en el lugar más importante ya que las personas son las que hacen el

software. La intención de mejora continua que tienen las personas en el proyecto del producto y del proceso.

- Las personas son lo más importante, donde se prioriza que haya comunicación, motivación, confianza, excelencia y autogestión. Ya que son éstas las que construyen el software.
- Debajo encontramos el pie, en el cual tenemos la mejora continua. Es el triángulo de resultados, teniendo como pie a la calidad y al valor.
- Del otro lado de la base podemos encontrar el alcance, el cual nos da cuáles serían las restricciones de nuestro producto, y encontramos el costo y el tiempo, los cuales suelen ser grandes limitantes a la hora de desarrollar software.



FILOSOFIA LEAN 02-Resumen completo

Requerimientos ágiles

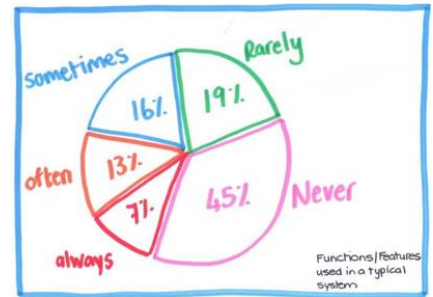
- El enfoque ágil no solo plantea una gestión diferente del proyecto sino también una definición de requerimientos distinta.
- En la gestión ágil, el **valor de negocio** y la construcción del producto correcto son fundamentales.
- En lugar de especificar todos los requerimientos desde el inicio, el desarrollo de requerimientos está enfocado en trabajar en conjunto con el cliente.
- Los requerimientos se expresan en forma de **historias de usuario**, que son breves descripciones de los requisitos del cliente que se escriben en lenguaje natural y se enfocan en las necesidades del usuario. Estas historias se utilizan para definir el alcance de cada iteración del proyecto y se priorizan en función del valor que aportan al producto.
- Se plantea que los requerimientos se irán descubriendo a medida que avanza el proyecto y se busca definir "**sólo lo suficiente**" para comenzar a trabajar con lo mínimo necesario y obtener retroalimentación lo antes posible para mejorar continuamente.
 - Todos los productos de software tienen un desperdicio significativo, esto quiere decir que, de todas las funcionalidades desarrolladas, es muy alto el porcentaje de aquellas que el cliente/usuario no usa.
 - La gestión ágil de requerimientos intenta contrarrestar esta situación, priorizando solo aquellas funcionalidades que aportan valor al cliente.
 - Se utiliza el concepto de Product Owner como una figura clave en el proceso de desarrollo quien es el responsable de identificar las necesidades y prioridades del negocio y, de priorizar los requerimientos del producto
 - Estos requerimientos se organizan en una lista priorizada y ordenada, llamada Product Backlog, donde los requerimientos más importantes se

encuentran en la parte superior de la lista, y los menos importantes en la parte inferior.

- Los requerimientos que están en el tope de la lista son aquellos en los que vamos a trabajar con mayor nivel de detalle y a medida que voy terminando con esos requerimientos, continuo con los que siguen en orden decreciente.
- A medida que avanzamos, hay requerimientos que pueden ser removidos, otros que pueden subir y tomar un mayor valor en la pila y otros que pueden bajar o bien pueden agregarse requerimientos a la pila.

BRUF – Big Requirements Up Front

La imagen adjuntada expresa la frecuencia con la que los usuarios utilizan determinadas funcionalidades de un software. A lo que se apunta con este concepto de BRUF es a notar que, en una primera instancia de un proyecto de software, desarrollar sólo unas pocas funcionalidades de gran valor e importancia para el cliente, puede cubrir gran parte de sus necesidades, a medida que se desarrolla el resto de los requerimientos.



Product Backlog

- El Product Backlog es una lista priorizada de características y requerimientos del software.
- Es el Product Owner quien se encarga de definir esta lista ya que es él quien tiene en claro cuáles son sus necesidades y requerimientos principales.
 - El PO se va a encargar de decidir qué requerimientos necesita primero según el valor que éstos tengan para el negocio.
- En contraposición con el enfoque tradicional, donde teníamos un cliente que no estaba dispuesto a comprometerse y dejaba la priorización de los requerimientos al equipo que hace el software.

Requerimientos Just in Time

JUST IN TIME

Analizo cuando lo necesito, no antes ni después (que no se haga tarde). Analizo con detalle un requerimiento cuando tenga el suficiente valor.

Es un concepto que se usa fundamentalmente para **evitar desperdicios** (que en este contexto sería invertir tiempo en especificar requerimientos que después cambian). El producto no va a estar especificado 100% desde el principio, vamos a ir encontrando requerimientos y describiéndolos conforme haga falta.

Lo que nosotros tenemos que entregarle al cliente es valor de negocio, no características de software. El software es el medio por el cual nosotros le entregamos valor de negocio.

Fuente de requerimientos

- Los requerimientos ágiles, basados en el principio de que el mejor medio de comunicación es el cara a cara, promueven que los requerimientos del software se obtienen conversando cara a cara con el cliente.
- En los ambientes tradicionales, los requerimientos se encuentran especificados en un documento de especificación de requerimientos (ERS) y el cliente luego lo lee para validarlo.

- Los agilistas no escriben una ERS, pero lo compensan con la disponibilidad del cliente para resolver las dudas. Esto permite reducir la cantidad de documentación formal.

Momento de la captura de requerimientos

- En el enfoque tradicional, los requerimientos son definidos al principio del proyecto. El flujo de trabajo de requerimientos tiene un gran peso en la fase inicial y de elaboración.
- En el enfoque ágil, los requerimientos son relevados continuamente durante todo el proyecto.

Contenedor y persistencia de los requerimientos

- El enfoque tradicional, la ERS contiene los requerimientos a lo largo del ciclo de vida del producto todo el tiempo. En caso de que surjan cambios, es necesario actualizar la ERS.
- En el enfoque ágil, se utiliza al Product Backlog como un contenedor transitorio de los requerimientos, ya que una vez implementados estos se “persisten” en el producto funcionando.

Tipos de requerimientos

DE NEGOCIO	Son los de más alto nivel, nos referimos a cuales son los requerimientos en términos de la visión del negocio
DE USUARIO	Tienen que ver concretamente con los requerimientos de usuario final
FUNCIONALES	Podríamos relacionarlo 1 a 1 con los casos de uso en la gestión tradicional
NO FUNCIONALES	Suelen ser los más ocultos, que el cliente no tiene en claro y debo especificarlos. Un requerimiento no funcional puede hacer que el software que estoy desarrollando no sirva (IMPORTANCIA FUNDAMENTAL)
DE IMPLEMENTACIÓN	Se suele decir que están dentro de los no funcionales, pero tienen que ver con cuestiones de restricción o implementación específicos.

En otras palabras, la gestión ágil de requerimientos implica trabajar junto a técnicos y no técnicos entendiendo las necesidades y el negocio para construir un software que de valor. Y luego, junto con los usuarios descubrir cuál es la mejor forma de satisfacer esas necesidades (Aquí aparece el Product Backlog). Entregada la característica, obtenemos feedback que nos va a ayudar a mejorar el Product Backlog si es necesario.

En este contexto, algunos conceptos relacionados con los principios ágiles que aplicamos son:

- **LOS CAMBIOS VAN A EXISTIR TODO EL TIEMPO DE FORMA CONSTANTE:** Sabemos que hay cambio, por lo que se busca gestionar los requerimientos orientados a poder aceptar el cambio. Si el Product Owner, el cliente o los stakeholders relacionados con el producto no plantean cambios es porque el producto no está siendo utilizado.
- **DEBEMOS TENER UNA MIRADA DE TODOS LOS QUE ESTÁN INVOLUCRADOS:** Todos los que tienen algo para decir del producto (Stakeholders).
- **EL USUARIO DICE LO QUE QUIERE CUANDO RECIBE LO QUE PIDIÓ.** Es por esto que las iteraciones cortas y las entregas continuas son esenciales, ya que permiten una retroalimentación y un enriquecimiento para futuras entregas y para una retro significativa
- **NO HAY TÉCNICAS NI HERRAMIENTAS QUE SIRVAN PARA TODOS LOS CASOS.** A veces se necesita trabajar con más prototipos o modelos, otras veces

con técnicas sencillas es suficiente, pero hay que ver en cada caso que es lo que conviene.

- LO IMPORTANTE NO ES ENTREGAR UNA SALIDA, UN REQUERIMIENTO, SINO ES ENTREGAR UN RESULTADO, UNA SOLUCIÓN DE VALOR. Vamos a apostar a entregar una solución que le de valor al cliente, con esto nos referimos a algo que sea útil para el negocio o el cliente minimizando el desperdicio.

Principios ágiles relacionados a los requerimientos ágiles (Solo enunciamos los MÁS relacionados)

1. La prioridad es satisfacer al cliente a través de releases tempranos y frecuentes (2 semanas a un mes)
2. Recibir cambios de requerimientos, aun en etapas finales.
4. Técnicos y no técnicos trabajando juntos todo el proyecto: El PO es parte del equipo, porque si los no técnicos no son parte del equipo del proyecto, difícilmente pueda identificar qué da valor al negocio
6. El medio de comunicación por excelencia es cara a cara.
11. Las mejores arquitecturas, diseños y requerimientos emergen de equipos auto organizados.

User Stories

- Son una técnica para trabajar requerimientos de usuario en ambientes ágiles.
- La US contiene una descripción corta de una funcionalidad que se espera del producto valuada por un usuario del sistema.
- Funciona como un recordatorio para el equipo de desarrollo y el Product Owner de que tienen que conversar acerca de la necesidad en términos de negocio para la construcción de una característica del producto que aporte valor. En otras palabras, es un mecanismo para diferir una conversación.
- Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario.
- Las US son escritas por el Product Owner, quien también las prioriza en el Producto Backlog.
- Es un requerimiento a nivel de usuario, no al nivel de sistema, por lo que se encuentra a un nivel de abstracción más elevado. Es un ítem de planificación que no acompaña al producto durante todo el ciclo de vida. Para eso está el manual de usuario.
- Las US se consideran verticales dentro del producto, ya que pueden incluir aspectos desde diseños de interfaces hasta diseños de tablas en la base de datos.
- Se dice que son multipropósito, debido a que cumplen diferentes funciones en el desarrollo de un software:
 - Representan una necesidad de usuario, de forma no detallada.
 - Conforman una descripción del producto.
 - Son utilizadas como ítems de planificación.
 - Se utilizan como recordatorios de futuras conversaciones acerca del producto que se desea construir.
 - Junto con el código ejecutable, permiten documentar el producto.

La problemática que motivó en la definición de esta técnica es la dificultad de comunicar lo que hará el sistema a todas las personas afectadas.

La definición de los requerimientos es lo que más afecta la calidad del sistema final y es lo más complejo de definir con detalle debido a la incertidumbre y cambio constante que sufren, que se da por el problema de comunicación.

Partes de la User Story (compuesta por tres partes)

Tarjeta: es una descripción de la historia, utilizada para planificar y como recordatorio.

<<Frase Verbal>>
Como <<nombre del rol>> yo puedo <<actividad>> de forma tal que <<valor de negocio>>

- Nombre del rol: representa quien está realizando la acción o quien recibe el valor de la actividad.
- Actividad: representa la acción que realizará el sistema.
- Valor de negocio: es la más importante y representa el por qué es necesaria la actividad. Esto justifica la razón de que el PO escriba las US. Permite priorizar el desarrollo.
- Frase verbal: no es obligatoria, es una forma corta de referenciar la US. Representa la funcionalidad que se expresa en la User Story.
- Criterios de aceptación: son diferentes criterios sobre la actividad que se necesita implementar, que permiten definir los límites de la U.S. para que el equipo tenga una mejor visión de esta. Además, facilitan las tareas de desarrolladores y testers para probar la funcionalidad. Se deben redactar independientes a la implementación (alto nivel de abstracción).
- Pruebas de aceptación: acompañan a la tarjeta en la parte trasera, y es donde se capturan todos los detalles (que se manifiestan en la conversación) de la User, permitiendo determinar cuándo una historia está completa. Esto servirá a los desarrolladores para probar si la implementación ha sido realizada de forma correcta y completa. Las pruebas pueden agregarse o quitarse en cualquier momento.

Conversación: Es la parte más importante de la US, ya que explicita la comunicación entre el PO y el Equipo de desarrollo que permite compensar la falta de especificación y detalle. El acuerdo por sobre la negociación del contrato y el principio de técnicos y no técnicos trabajando juntos durante todo el proyecto están asociadas a esta parte de la US. El Product Owner es parte del equipo, por lo cual se espera responsabilidad, compromiso y confianza de su parte para hacerse cargo y llegar a un acuerdo común con el equipo sobre lo definido acerca de la funcionalidad. También es posible dejar la conversación persistente en grabaciones, minutas o softwares de gestión que posibiliten la conversación. De todas maneras, la conversación se materializa de forma distribuida en la tarjeta y en las pruebas de aceptación.

Confirmación: Definición de un acuerdo entre el PO y el Equipo para demostrar que la característica del producto realmente se implementó. Conformar a la definición de las pruebas de aceptación. El PO decide si las pruebas de aceptación son válidas. Pruebas que se usan para determinar cuándo una US está completa.

Product Backlog y las User Stories

- Es una pila de PBIs (Product Backlog Items) con sus correspondientes estimaciones de tamaño, en un formato determinado. En la parte más alta de la pila, se ubican las U.S. de mayor prioridad (que son aquellas U.S. con granularidad

final), y las de menor prioridad (granularidad gruesa, sobre las cuales no se tiene mucho detalle ya que no se consideran indispensables) al fondo. En cualquier momento se puede cambiar esa priorización, según cómo lo indique el PO, quien es el encargado de realizar el proceso de priorización.

- En cualquier momento del proyecto se van agregando nuevas U.S. y actividades a realizar, según se vayan detectando nuevos requerimientos.
- Para quitar una U.S. del Product Backlog, se planifica una iteración y se decide, según la velocidad de trabajo del equipo [Story Points / iteración], cuántas U.S. de la pila se quitarán para implementarlas en la próxima iteración.
- Debido a estas dos situaciones (agregar y quitar ítems), nunca encontraremos el 100% de los requerimientos de usuario en el Product Backlog.

Porciones verticales

Las US son porciones verticales, es decir, para poder agregar valor al cliente debemos tener un poco de interfaz, un poco de lógica de negocio y otro poco de base de datos. Debemos diseñar el pedacito de cada cosa que nos hace falta para que una determinada característica funcione.

Relación con la arquitectura en capas

La User es una porción vertical, la cual abarca todas las capas de arquitectura: capa de datos, lógica de negocio y capa de presentación. Esto permite entregar una característica terminada al cliente.

Tamaño

- Es recomendable tener una gran cantidad de historias con tamaños relativamente pequeños, que tener historias muy grandes.
- El tamaño de las U.S. se mide en Story Points.
- Hay 3 aspectos que determinan el tamaño de una U.S.:
 - Complejidad: hace referencia a una dificultad inherente a la funcionalidad, que no permiten dividirla en U.S. más pequeñas. Por ejemplo, algoritmos complejos, cálculos, gran cantidad de campos, muchas validaciones, demasiados filtros, etc.
 - Esfuerzo: es el tiempo en horas de trabajo que requerirá la implementación.
 - Incertidumbre:
 - Técnica: cuando se tiene dudas en el dominio de la solución, por ejemplo, el uso de una cierta tecnología no muy conocida por el equipo.
 - De negocio: cuando hay incertidumbre respecto a cómo interactuará el usuario con el sistema, lo cual es causado por falta de conocimiento del dominio del negocio.

Story Points

Son unidades de medida relativas (no importa el valor en sí, sino su comparación), las cuales miden el tamaño de un producto, NO de una medida basada en el tiempo.

Esto se debe a que estimar basándonos en el tiempo, conduce a muchos errores. Lo relativo surge a partir de que se comprobó que las personas no saben estimar en términos absolutos, sino que les resulta útil la comparación de tamaños.

Niveles de abstracción

- User Story: cumple con el criterio de Listo (DoR) para ingresar a una iteración.
- Épica: es una User muy grande. El tamaño es relativo a si entra en una iteración o no, ya que las User se empiezan y se terminan en la iteración. Si la User no entra en la iteración es considerado una épica. Tienen la misma sintaxis que la User. Una épica se suele identificar también cuando no cumple con el modelo de calidad de INVEST.

La épica debe ser dividida en US más pequeñas. Sirven para aquellos requerimientos que no son prioridades en las primeras iteraciones, y basta con tener sólo una aproximación general de las funcionalidades requeridas. Llegado el momento de su implementación, se deberán dividir en historias de usuario más pequeñas, para poder planificar su implementación en sucesivas iteraciones.

- Temas: son un conjunto de User Stories agrupadas, debido a que conceptualmente están relacionadas. Se las agrupa para tratarlas como una entidad simple a la hora de estimar o planificar.

Modelado de roles

- Se utilizan las tarjetas de rol de usuario, aquí se anotan las características en general que va a tener el usuario.
- El tema crítico de la funcionalidad está relacionado con quien va a utilizar dicha funcionalidad. Por esta razón existe una tendencia por ocuparse de quien va a usar el sistema, especificando los roles de usuario donde definimos las características del usuario respecto al producto y en relación con su perfil genérico.
- Técnicas adicionales
 - Personas: esta técnica busca ser más específica acerca de la descripción de la persona concreta que va a utilizar el sistema. Este análisis es exhaustivo y por lo tanto complejo. Si los roles son clases, las personas serían los objetos.
 - Personajes extremos: que personas en términos extremos podrían llegar a utilizar el software.
 - Proxies (Usuarios Representantes): es un representante de todos los usuarios. Debe tener capacidad de tomar decisiones desde el negocio.

Criterios de aceptación

- Los criterios de aceptación son aquellas cuestiones que le definen límites a la User Story.
- Ayudan a que el PO responda los criterios que necesita para que la US provea valor al negocio y también a que el equipo tenga una visión compartida de la US.
- No contienen cuestiones de implementación, tienen que ver con la perspectiva del usuario; siempre definen una intención, no pensamos en la solución, sino en lo que de alguna manera el Product Owner nos manifiesta en su intención.
- Los criterios de aceptación ayudan a su vez a los desarrolladores a saber cuándo parar de agregar funcionalidad y a los testers a derivar las pruebas pertinentes.
- Los criterios de aceptación buenos son aquellos que definen una intención y no una solución, que son independientes de la implementación y que son relativamente de alto nivel.
- Son las consideraciones que define nuestro usuario a la hora de hablarnos de cómo imagina él esa funcionalidad.
- No hay limitación en la cantidad de criterios.
- No van detalles dentro de los criterios de aceptación

PUEDE	Opcional
DEBE	Obligatorio

Detalles como:

- El encabezado de la columna se nombra "Saldo"
- El formato del saldo es 999.999.999,99
- Debería usarse una lista desplegable en lugar de un Checkbox.

Estos detalles que son el resultado de las conversaciones con el PO y el equipo puede capturarlos en dos lugares:

- Documentación interna de los equipos

Pruebas de aceptación

- Se encuentran en el dorso de la tarjeta, complementan la historia, y también expresan detalles de la conversación.
- Nos dan la confirmación de que finalmente lo que nosotros estamos haciendo, si todas las pruebas que nosotros definimos las pasan.
- Profundiza lo que estamos definiendo en la US y nos ayuda a terminar de definir cómo implementar la US. Agregamos también lo que esperamos que suceda cuando ejecutamos la historia de usuario.
- Entre paréntesis agregamos PASA o FALLA refiriéndonos al resultado del test.

Definición de listo o criterio de ready

- Es una medida de calidad que determina si la User Story está en condiciones de entrar a una iteración de desarrollo.
- Para que la US pueda ser implementada, mínimamente debe satisfacer el INVEST Model.

INVEST Model

Es un modelo de calidad que nos ayuda a verificar si la User cuenta con las características de la calidad mínima para satisfacer la definición de Ready y poder ingresar en una iteración de desarrollo.

I: INDEPENDIENTE	Que no dependa de otra US, poder incluirla en una Sprint porque total no depende de otra US, puede ser implementable en cualquier orden. No me afecta a otra US mover mi US dentro del Product Backlog
N: NEGOCIABLE	Se negocia el QUÉ, no el CÓMO; por esto es que decimos que en los criterios de aceptación no ponemos detalles de implementación, lo que negociamos con nuestro cliente es el QUÉ. Si la user define el COMO está mal definida
V: VALUABLE	Debe tener un valor concreto para el cliente
E: ESTIMABLE	Tengo que poder definir cuánto esfuerzo me va a llevar hacer la US, para ayudar al cliente a armar un ranking basado en costos.
S: SMALL	La US tiene que entrar en una Sprint. Para saber si es small, tengo que poder estimarla.
T: TESTEABLE	Tengo que poder demostrar que fueron implementadas.

DoD (Definition of Done) – Definición de Hecho

- Determina si la US está terminada.
- Son los criterios que establecen cuándo una User Story ha sido implementada de forma correcta y completa, de forma que puede ser mostrada al Product Owner, para que éste decida si se pasa a producción o no.
- En la práctica consta de un Checklist de condiciones que deben cumplirse.
- Puede ocurrir que una organización tenga un estándar de DoD, entonces los miembros del equipo deberán adaptarse a ese estándar. O bien, que la

organización no posea un estándar, con lo cual el equipo deberá confeccionar y crear una Definición de Hecho adecuada para el producto.

Spikes

- Son un tipo especial de US que se producen por la incertidumbre que la misma presenta, la cual imposibilita que pueda ser estimada y por lo tanto no cumple con la definición de listo.
- Sirven para quitar riesgo o incertidumbre de otro requerimiento, de otra US o de alguna faceta del proyecto que nosotros queremos investigar. Son específicamente creadas para esto.
- Una vez resuelta la incertidumbre, la Spike se convierte en una o más US.
- Pueden ser:
 - Técnicas: Son utilizadas para investigar enfoques técnicos en el dominio de la solución. Cualquier situación en la que el equipo necesite una comprensión más fiable sobre alguna tecnología a aplicar antes de comprometerse a desarrollar una nueva funcionalidad en un tiempo fijo.
 - Funcionales: Utilizada cuando hay cierta incertidumbre respecto de cómo el usuario interactúa con el sistema, usualmente son mejor evaluadas con prototipos para obtener retroalimentación de los usuarios involucrados.

LINEAMIENTOS A TENER EN CUENTA

Diferir el análisis detallado tan tarde como sea posible, lo que es justo antes que el trabajo comience. Hasta ese entonces, los requerimientos se capturan en forma de User Stories (como épicas o temas) y se agregan al Product Backlog según el valor que aportan al negocio

Las US no son requerimientos de software, no necesitan descripciones exhaustivas de la funcionalidad del sistema

Tener en cuenta que en la US se hace un paso a la vez, **evitar usar la palabra "y"**, usar palabras claras en los criterios de aceptación y sobre todo y más importante, **escribir la user story desde la perspectiva del usuario**.

Recordar que la parte invisible de la user (CONVERSACION) es la más importante.

Investment Themes (Temas de Inversión)

- Representan un conjunto de iniciativas o propuestas de valor que conducen la inversión de la empresa en sistemas, productos, aplicaciones y servicios con el objetivo de lograr una diferenciación en el mercado y/o ventajas competitivas.

Estimación en Tradicional

- Estimar es el proceso de obtener una aproximación sobre una medida con el objetivo de predecir la completitud y gestionar los riesgos en el contexto de un proyecto.
- McConell "una estimación en el contexto del software es una predicción de cuánto tiempo durará o costará un proyecto". Es considerada una de las actividades más complejas en el desarrollo de software.
- Estimar NO es planear y no necesariamente nuestro plan tiene que ser lo mismo que lo estimado. Si bien la estimación sirve como base para planificar, los planes no tienen que seguirla, las estimaciones no son compromisos.
- Al planear también incluimos otras consideraciones como cuestiones que tienen que ver con el objetivo del negocio y estas hacen la diferencia con la estimación de entrada. Eso sí, debemos tener en cuenta que mientras mayor diferencia haya entre lo estimado y lo planeado mayor riesgo va a haber.

¿Por qué estimamos?

- Antes de que el proyecto comience, el líder del proyecto y el equipo de software deben estimar el trabajo que habrá de realizarse, los recursos que se requerirán y el tiempo que transcurrirá desde el principio hasta el final. Las estimaciones requieren: Experiencia, Acceso a buena información histórica (métricas), El valor para comprometerse con predicciones cuantitativas cuando la información cualitativa es todo lo que existe.
- Existen distintas **técnicas de estimación** que nos ayudan a disminuir/acotar esa incertidumbre.
- Cada técnica tiene sus propias ventajas y desventajas, y es importante seleccionar la técnica adecuada para cada proyecto en función de sus requisitos y características específicas. Además, es importante realizar una revisión y actualización constante de la estimación a medida que se avanza en el proyecto para garantizar que se ajuste a la realidad del desarrollo del software.
- Métricas básicas para un proyecto de software (tamaño del producto, esfuerzo, tiempo/calendario, defectos)
- Lo primero que estimamos es el TAMAÑO DEL SOFTWARE:
 - Con tamaño del software nos referimos a que tan grande va a ser el trabajo que vamos a hacer. Para esto utilizamos la técnica de CONTAR: Contar funcionalidades, Contar requerimientos, Contar la cantidad User Stories, Contar la cantidad de Casos de Uso.
- Otra estimación que realizamos es el ESFUERZO:
 - Con esfuerzo nos referimos a la cantidad de horas lineales que voy a necesitar para construir el software, no incluyo qué personas lo hacen, ni calendario ni tampoco me paro a especificar si las actividades son paralelas o secuenciales, solo CUENTO, en estimación, cuantas horas lineales de desarrollo voy a necesitar.
- Otra estimación es el CALENDARIO
 - Es calendarizar el esfuerzo y proponer una fecha de entrega.
- Una vez que logramos estimar tamaño, esfuerzo y tiempo calendario, vamos a empezar a trabajar con COSTOS y PRESUPUESTOS.
 - Depende directamente del tamaño y del esfuerzo que se necesite para el desarrollo del software, por eso es lo último que se estima.
 - Es muy útil para darle una idea al cliente de cuál es el costo aproximado del proyecto.

Métodos y Técnicas utilizados para estimar (Los métodos adentro tienen técnicas)

Se recomienda utilizar diferentes métodos de estimación y luego contrastar.

- **Met basados en la experiencia:** se basa en la experiencia pasada del equipo en proyectos similares para determinar la estimación de esfuerzo y tiempo requerido para un nuevo proyecto.
 - Datos históricos(no acorde a ágil):
 - Implica comparar un proyecto nuevo con uno anterior que me nutra y me permita hacer la estimación de mi proyecto.
 - Necesito tomar como entrada a mis estimaciones son el tamaño, el esfuerzo, el tiempo y los defectos.
 - Juicio de experto (más utilizado):
 - Se basa en la experiencia y conocimiento de expertos en la materia para determinar la estimación de esfuerzo y tiempo requerido para un proyecto de software.

- Se basa en la subjetividad y experiencia de los expertos, y las estimaciones pueden variar significativamente dependiendo de la experiencia y conocimiento de los expertos involucrados.
- Puro:
 - Un experto estudia las especificaciones y realiza una estimación. Esta estimación se basa fundamentalmente en la experiencia del experto. (problema si se va de la org)
 - Se puede utilizar el “método de optimista, pesimista, habitual”, en el que se le pregunta al experto sobre 3 valores distintos y se aplica una fórmula para calcular la estimación.
- Wideband Delphi (se usa en tradicional y agil):
 - Un grupo de personas se reúne con el objetivo de converger a una estimación común, tanto en esfuerzo como en duración.
 - Hay un coordinador que va a reunir las estimaciones individuales de cada uno de los expertos cuando les dieron las especificaciones, se opina sobre las estimaciones ajenas de manera anónima, se vuelve a discutir, revisan, las vuelven a enviar al coordinador y se repite hasta que todos estén de acuerdo de forma razonable.
 - Base del poker planning.
- Analogía:
 - Se comparan los factores a estimar de forma relativa con respecto a las estimaciones de otros proyectos similares.
 - Es un método que tiene mucho error dado que el conocimiento en un proyecto no es extrapolable a otro.
- **Met Basados exclusivamente en los recursos:** se basa en la cantidad y el tipo de recursos (personas, tiempo, herramientas, etc.) necesarios para llevar a cabo un proyecto.
- **Met Basados en la capacidad del cliente o exclusivamente en el mercado:** ponen el foco en el cliente y realizan las estimaciones en función de su capacidad de pago. En este método no se evalúa al producto que se está intentando vender, sino a quien se lo vende.
- **Met Basados en los componentes del producto o en el proceso de desarrollo:** se basa en la identificación y descomposición de los componentes del software y del proceso de desarrollo para estimar el esfuerzo y tiempo requerido para cada componente. Luego, se suman las estimaciones para obtener una estimación total del proyecto.
- **Met algorítmicos:** se basa en modelos matemáticos y estadísticos para determinar la estimación de esfuerzo y tiempo requerido para un proyecto.

Problemas de estimaciones tradicionales

- Estimar demasiado pronto.
- Resistencia a las reestimaciones.
- Estimaciones que buscan precisión.
- Estimaciones hechas por gente distinta a la que hace el trabajo.
- Estimaciones que pretenden ser absolutas (inflexibles).

Estimaciones en ambientes ágiles

- Son presunciones numéricas asociadas a una probabilidad de que cierto evento ocurra.
- La filosofía ágil plantea que quien estima, es quien debe hacer el trabajo, con lo cual es el equipo de desarrollo quien realiza las estimaciones, de forma colectiva (menos el PO). De esta manera, la estimación se ajusta mejor, debido a que se generan debates entre los miembros del equipo, lo cual puede abrir puntos de vista no detectados por otros.
- Las estimaciones se tienden a confundir con compromisos o planificación. Uno planifica basándose en la estimación, pero no son lo mismo.
- No es una instancia única, ya que a medida que se avance el proyecto, más información vamos a tener y más eficiente van a ser nuestras estimaciones.
- Existe una serie de consideraciones importantes:
 - Son medidas relativas no absolutas: ya que las define un equipo y no son comparables entre distintos equipos.
 - No es una medida basada en el tiempo: Las estimaciones basadas en base al tiempo son más propensas a errores debido a factores externos como las habilidades de las personas, el conocimiento del dominio, la experiencia, etc.
 - Las personas no saben estimar en términos absolutos, somos buenos comparando: lo hacemos a través de la definición de una User Story Canónica (de base) no necesariamente la más chica pero una que defina en base a que vamos a definir los Story Points.
 - Comparar es generalmente más rápido.
 - Se obtiene una mejor dinámica grupal y pensamiento de equipo más que individual.
 - Se emplea mejor el tiempo de análisis de las Stories.
- Para poder estimar una US debo estimar el tamaño, el tiempo y el esfuerzo necesario de la misma.
 - El **tamaño** es una medida de la cantidad de trabajo necesario para producirla o completarla, y a su vez indica cuan compleja y cuán grande es. Es independiente de quien haga.
 - Las estimaciones basadas en base al **tiempo** son más propensas a errores debido a factores externos como las habilidades de las personas, el conocimiento del dominio, la experiencia, etc. Un trabajo puede requerir un esfuerzo de x horas, pero no se puede asegurar que para tal día va a estar terminado. Ya que en medio pueden suceder cosas que evitan que cumpla mi objetivo.
 - No debemos confundirnos, tamaño NO es **esfuerzo**, ya que esfuerzo tiene que ver con horas lineales. Depende específicamente de la persona que lo va a llevar a cabo

Planificación

- En ambientes ágiles, es necesario que quien planifica el proyecto de desarrollo de software, sea capaz de analizar el contexto del proyecto cada cierto período y hacer aquellos ajustes que sean necesarios, para encauzar los esfuerzos de manera correcta, y poder lograr los objetivos planteados.
- “Planning is everything. Plans are nothing” → esto apunta a que lo importante de planificar, es el proceso en sí.

- No tener una documentación exhaustiva. Sí debe quedar registrado de alguna manera, idealmente bajo templates, pero sólo para servir de ayuda de memoria. Lo importante es el proceso.
- Los equipos ágiles afrontan esta situación, planificando al menos en 3 niveles (release, iteración, día):
 - Planificación de estrategia
 - A que productos le voy a dar mas trascendencia, como los voy a hacer evolucionar, etc.
 - Planificación de portfolio
 - Conjunto de productos de la misma línea, por ejemplo en el office tenemos un portfolio de word, excel, PowerPoint, etc.
 - Planificación de producto
 - Cuanto tiempo y como voy a hacer para lograr construir el producto final que yo espero.
 - Si bien sabemos que el mismo va cambiando, es necesario tener una visión y una evolución del mismo. Como también un roadmap de como llegar al producto que quiero construir.
 - Planificación de Release
 - Se da al inicio del proyecto, donde se analiza y determina qué US, épicas o Temas serán desarrollados en una nueva entrega (Release) del producto.
 - Se va actualizando mientras el proyecto transcurre, de manera que siempre refleja las expectativas actuales sobre qué se incluye en la entrega.
 - Planificación de Iteración (lo mismo que el sprint) → tiene como salida un incremento del producto
 - Este proceso se da al comienzo de cada iteración, donde el PO identifica aquellas US de mayor prioridad que deberán ser implementadas en la iteración, basándose en la iteración anterior ya finalizada.
 - Planificación del día
 - Permite coordinar y sincronizar el trabajo del equipo, normalmente en reuniones diarias. Allí sólo se discuten las tareas a realizar durante el día, haciendo foco en las actividades que deberán ser desarrolladas para lograr cumplir con una tarea específica.

Tamaño (Mas arriba explica como se calcula: complejidad, esfuerzo, incertidumbre)

- Medida de cuán compleja y grande es una US, además de cuánto trabajo es requerido para hacer o completar una Feature/US
- También el tamaño es función de la incertidumbre de dicha US (falta de información)
- Formas de estimar el tamaño:
 - Número del 1 al 10.
 - Talles de remeras: S, M, L: suele ser utilizado como escala de estimación para los ítems del Product Backlog.
 - Serie exponencial de base 2: 2, 4, 8, 16, 32, 64, etc.
 - Serie de Fibonacci: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, etc.

¿Para qué se estima?

- En un primer momento para determinar cuántas Users se puede comprometer el equipo de trabajo a terminar en una iteración.

- Al finalizar la iteración, contamos los puntos de historia correspondiente a las Users aceptadas por el Product Owner y obtenemos la velocidad del sprint, la cual es una de las métricas fundamentales en ambientes ágiles.

¿Cómo se estima la duración del proyecto?

Si la estimación se realiza utilizando Story Points para medir la complejidad relativa de las User Stories, para determinar la duración de un proyecto se realiza la derivación tomando el número total de story points de sus US y dividiéndolas por la velocidad del equipo.

Métodos de estimación

- Poker Planning o Poker Estimation
 - Resulta de la conjunción de diferentes métodos como el de juicio experto, analogía y desagregación, basado en que 4 ojos ven más que 2.
 - En contraposición al método de juicio experto tradicional, asume que las personas que desarrollan el producto deben ser aquellas que estimen.
 - El equipo estima su propio trabajo y los miembros opinan sobre lo mismo, donde se mezcla la experiencia y el conocimiento de cada uno con la posibilidad de compartir con el resto del grupo.
 - El Product Owner participa en la planificación, pero no realiza estimaciones, suele ser el moderador, pero no es necesario que sea así siempre.
 - Escala: Dado que la complejidad en el software incrementa de forma exponencial, se decide adoptar la serie de Fibonacci
 - Valor 1 o 1/2: son los valores más bajos y se aplican a una funcionalidad pequeña.
 - Valor 2- 3: funcionalidad pequeña a mediana.
 - Valor 5: funcionalidad media.
 - Valor 8: funcionalidad grande, donde nos preguntamos si es posible dividir.
 - Valor 13 o más: necesariamente hay que dividir ya que no ingresa en el sprint.
 - Procedimiento:
 - Se define una lista de actividades, módulos, casos de uso, Users stories, etc.
 - Se acuerda cuál de las anteriores es la que tiene menor grado de incertidumbre y se la define como canónica asignándole 1 o 2 story point.
 - Cada miembro elige una carta con el valor de la estimación del tamaño de la U.S y se espera a que todos decidan.
 - Utilizando juicio experto, cada integrante del equipo de desarrollo expone su valor de estimación de forma simultánea, por medio de las cartas, dándolas vuelta a las que estaban boca abajo. En caso de que la estimación difiera, el mayor y el menor estimador justifican sus estimaciones para conocer sus opiniones.
 - En caso de que la primera vuelta existiese diferencias considerables, se realiza nuevamente la ejecución del paso 3.
 - Si las estimaciones no convergen, entonces se llega a una estimación intermedia o promedio.
 - Consideraciones

- Para realizar una estimación se debe tener en cuenta el criterio de Done y no solo la tarea de programación.
- El tiempo para realizar las estimaciones, como cualquier otra actividad en los ambientes ágiles, es time boxing.
- Las US por estimar son las menos posibles, en orden con el principio de diferir compromisos.

Estimaciones en Lean En Lean

El cual es más extremo que las metodologías ágiles, asume que las estimaciones son opcionales y en algunas ocasiones pueden convertirse en un desperdicio dado que tienen una alta probabilidad de no coincidir con la realidad, dado que cualquier variable que se modifica, modificara a la estimación también.

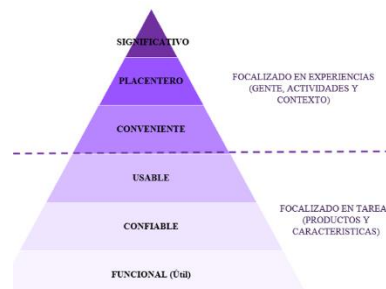
Frameworks de SCRUM a nivel de equipo y escala (VA EN OTRA FILMINA)

- Scrum es un marco ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos.
- En pocas palabras, Scrum requiere un Scrum Master para fomentar un entorno donde:
 - Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.
 - El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
 - El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
 - Repetir

Gestión de productos de software

Evolución de los productos de software

- Cuando vamos a desarrollar un software la mayoría de las veces se empieza a construir un producto sobre la base de lo funcional, que haga lo que tiene que hacer.
- Luego, el software debe ser confiable donde los usuarios no corran peligro usando el producto y los resultados que este producto me da yo pueda depender de ellos sin ningún problema.
- Luego, la usabilidad tiene que ver con aspectos de la disciplina UX/UI, donde el usuario podrá lograr sus objetivos productivamente usando este producto, estando a gusto con su uso. Tiene que ser productivo, no es lo mismo que la utilidad.
- En el tope de la pirámide aparecen otros aspectos relacionados con otras cuestiones de la visión del producto que son difíciles de alcanzar.



No solo está asociado con lograr la satisfacción de los clientes, la obtención de dinero o la masividad y popularidad de la aplicación, sino también con una visión de cambiar el mundo o nuestra diaria.

Sabemos que muchas veces se invierte mucho tiempo en construir características del software que no se usan o se usan muy rara vez. Esto implica un esfuerzo innecesario en algo que no se va a utilizar o vender. Es muy importante entonces, eliminar el desperdicio para poner nuestro esfuerzo en aquello que sea útil.

El desafío tiene que ver con cómo hacemos para construir un producto de software donde desechamos el desperdicio y logramos abarcar los aspectos que estamos persiguiendo cuando creamos el producto de software.

Nos vamos a centrar en desarrollar el mínimo producto o mínima característica para saber si el producto que vamos a construir va a cubrir las expectativas del usuario o las propias al desarrollar el producto.

Mínimo Producto Viable

- Es un concepto de Lean Startup que enfatiza el impacto del aprendizaje en el desarrollo de nuevos productos.
- Se define como la versión de un nuevo producto que permite a un equipo **recopilar la cantidad máxima de aprendizaje validado** sobre los clientes con el menor esfuerzo.
- **El objetivo es comprobar que la hipótesis del producto a construir funciona** o si tengo que hacer cambios. **La idea es iniciar un circuito de aprendizaje** a partir de la participación de los usuarios y la retroalimentación sobre si el producto cubre expectativas, sirve o si tengo que hacer cambios en las características incluidas.
- Puede suceder, que así como está definida la hipótesis sirva o puede ocurrir que cada potencial usuario de ese MVP cuando lo vea pida funcionalidades diferentes. Si con cada cliente potencial que prueba la MVP recibo un feedback distinto, quiere decir que no está tan claro el mercado en el cual este producto va a funcionar y significa que tengo que seguir trabajando sobre la visión del producto para encontrarlo.
- Otra alternativa que podría suceder es que, en algún punto, la hipótesis planteada tenga algunas variaciones. Y en este contexto, la hipótesis puede moverse o modificarse según la exigencia del mercado.
- Debemos redefinir el MVP en base al conocimiento adquirido, por lo que creamos un nuevo MVP2. Lo que hacemos es pivotar hasta encontrar específicamente cual es el producto que nosotros queremos llegar a construir. Este circuito de retroalimentación se vuelve hacer, pero con este nuevo MVP.

Una vez que encontramos el MVP exitoso, es decir que el feedback y la investigación son exitosas, surge el concepto de:

MMF (Mínimum Marketable Feature – Característica Mínima Comercializable)

- Se enfoca en definir los elementos mínimos que deben estar presentes en un producto o servicio para que sea comercializable.
- Ya no estoy parado en una hipótesis para validar si el producto tiene mercado o no, sino que estoy definiendo cual es la pieza mínima que voy a tener que construir y comercializar para que el producto sea rentable.
- El objetivo es asegurarnos de que este primer MMF sirve o no. **Con la diferencia de que ahora hablamos de una Feature, algo mucho más chiquito que el MVP.** Es lo mínimo que puedo sacar a la venta.

Para conjugar/combinar el MVP con la MMF, es a partir de la llamada:

MVF(Minimum Viable Feature - Característica Mínima Viable)

- Se trata de la característica mínima que se puede construir e implementar rápidamente, utilizando recursos mínimos, para probar la utilidad de la misma.

- Quiero validar cual es la característica que es la que hace la diferencia y que va a hacer en primera medida que compren mi producto.
- Es una feature que buscamos que por si misma ofrezca un valor agregado. Este es nuestro objetivo, ahora la hipótesis se centra en una característica en vez de un producto.
- Si la MVF resulta exitosa, se pueden desarrollar más MMF en esta área para tomar ventaja (ya que está comprobada).
- De alguna forma la MVF es una versión mini del MVP. Si seguimos trabajando en el contexto de validar las hipótesis y ver cómo vamos a trabajar con cada característica, cuando se logra ajustar el producto en el mercado, se combinan MMF y MVP según el nivel de incertidumbre del negocio o las áreas en las que se está enfocando

MRF (Minimum Release Feature – Características Mínimas del Release)

Se trata del release del producto que tiene el conjunto de características más pequeño posible. El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.

Relaciones concretas

MVP

- Versión de un **nuevo producto** creado con el **menor esfuerzo posible**
- Dirigido a un **subconjunto de clientes potenciales**
- Utilizado para obtener **aprendizaje validado**.
- Más cercano a los **prototipos que a una version real funcionando de un producto**.

MMF

- es la **pieza más pequeña de funcionalidad** que puede ser liberada
- tiene valor tanto para la organización como para los usuarios.
- Es parte de un MMR or MMP.

MMP

- Primer release de un MMR dirigido a **primeros usuarios** (early adopters),
- Focalizado en características clave que satisfarán a este grupo clave.

MMR

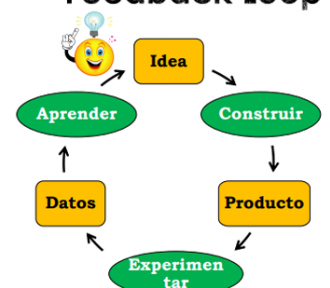
- Release de un producto que tiene el conjunto de características más pequeño posible.
- El incremento más pequeño que ofrece un valor nuevo a los usuarios y satisface sus necesidades actuales.
- **MMP = MMR1**

La fase construir del MVP

- Uno de los aspectos importantes de esta metodología o técnica es el ciclo de Feedback o de Aprendizaje.
- El éxito no es entregar un producto, el éxito se trata de entregar un producto (o característica de producto) que el cliente usará. La forma de hacerlo es alinear los esfuerzos continuamente hacia las necesidades reales de los clientes.
- The Build-Experiment-Learn feedback Loop permite descubrir las necesidades del cliente y alinearlas metodológicamente.

La idea es tener rápidamente y con el menor esfuerzo tener el MVP para poder validarlo y retroalimentar la hipótesis para entrar en el ciclo de aprendizaje. La fase experimentar pasa a ser clave al trabajar en la construcción del MVP. Aca empiezan a surgir distintos planteos al momento no solo de construir un MVP sino de construir en si un nuevo producto.

Build-Experiment-Learn Feedback Loop



Dilema: La audacia cero

- Debemos pensar en términos de la construcción de un producto en grande, aunque resulte más riesgoso. Si se pospone la experimentación con el MVP, van a surgir resultados como una gran cantidad de trabajo desperdiciado, pérdida de información esencial y el riesgo de construir algo muy poco significativo.
- Es por esto que debemos utilizar el MVP para experimentar (inicialmente, en silencio) con los primeros usuarios en el mercado verificando mi hipótesis probando todos los elementos disponibles comenzando por los más riesgosos.

Supuestos de “saltos de fe”

Los elementos más riesgosos del plan / concepto de un nuevo producto o startup se denominan supuestos de salto de fe. La mayoría de las personas no conocen una determinada solución pero una vez que experimentan la solución, no pueden imaginar cómo vivirían sin ella.



Componentes de un Proyecto de Sistemas de Información.

Proyecto

- Un proyecto de software es un esfuerzo temporal que requiere del acuerdo de un conjunto de especialidades y recursos para la creación de un producto, servicio o resultado único.
- Es una unidad organizativa para adaptar el marco teórico del proceso, dependiendo de las personas y recursos con los que se cuente.
- Define el proceso y tareas que se van a realizar, el personal que se encargará de las actividades y los mecanismos que se implementarán para valorar riesgos, controlar el cambio y evaluar la calidad.

Características

- Únicos: El resultado de un proyecto será único e irrepetible para cualquier otro proyecto, por más parecido o igual sean sus elementos componentes. Todos los proyectos por más similares que sean tienen características propias que los hacen únicos.
- Orientado a objetos:
 - No ambiguos: deben ser lo suficientemente claros para guiar el desarrollo del proyecto.
 - Medibles: es necesario medir el objetivo para determinar el avance sobre el proyecto.
 - Alcanzable: deben ser factibles de realizarse por el equipo (que el equipo pueda hacerlos).

- Duración limitada de tiempo: Esto implica que un proyecto tiene un principio y un fin, liberando los recursos y personas. Cuando se alcanzan los objetivos del proyecto, este llega a su fin.
- Conjunto de tareas interrelacionadas basadas en esfuerzo y recursos: Se definen las tareas, sus dependencias, los recursos y personas asignadas, permitiendo manejar la complejidad inherente de los sistemas.

Administración de proyecto de software

- Es la organización y coordinación de personas y recursos que permiten desarrollar un producto o servicio de acuerdo con el presupuesto, tiempo y funcionalidades acordadas con el cliente, asegurando que se entregue un producto o servicio de alta calidad.
- Es una parte fundamental para que el proyecto sea exitoso.
- Tiene dos grandes actividades:
 - Planificación
 - Monitoreo y control.
- Incluye:
 - Identificar los requerimientos.
 - Establecer objetivos claros y alcanzables.
 - Adaptar las especificaciones, planes y el enfoque a los diferentes intereses de los involucrados (stakeholders).
- Una buena gestión no puede garantizar el éxito del proyecto. Sin embargo, una mala gestión usualmente implica una falla en el proyecto.
- Un proyecto planificado puede fracasar también, lo importante de la planificación es el acto de planificar, no el resultado.

Metas más importantes de la administración de proyectos

- Entregar software al cliente en el tiempo acordado.
- Mantener los costos dentro del presupuesto general.
- Entregar software que cumpla con las expectativas del cliente.
- Mantener un equipo de desarrollo óptimo y con buen funcionamiento.

Atributos particulares o complejidades en la administración de proyectos de software

- El producto es intangible: No poder ver ni tocar el producto que se obtiene al ejecutar el proyecto dificulta la medición del progreso del proyecto.
- Los proyectos de software suelen ser excepcionales: Los conocimientos aprendidos en un proyecto de software son difícilmente extrapolables a otro proyecto, como validación de riesgos, aplicación de nuevas tecnologías, etc. El factor tecnológico es una de las causas.
- Los procesos de desarrollo son variables y específicos de la organización: A pesar del intento de estandarizar los procesos de desarrollo, la realidad es que no es posible predecir de manera confiable qué proceso es el adecuado para adoptar en el proyecto.

La triple restricción – enfoque tradicional

- Las variables de restricción de un proyecto son 3: tiempo, costo y alcance.



- El balance de estos tres factores afecta directamente la calidad del proyecto y **es responsabilidad del Líder de proyecto balancear estas variables.**
- El concepto de la triple restricción hace referencia a que no es posible fijar de forma arbitraria a las 3 variables, sino que será posible fijar 2 y la tercera se ajustará a lo definido. (No se negocia la calidad)

Alcance – ¿Qué trata de alcanzar? (Esta variable es la que primero se negocia con el cliente.)

- Son los requerimientos que se incluyen en el proyecto y definen la funcionalidad que tendrá el producto a entregar al cliente.
- En las metodologías tradicionales el alcance es lo primero que se determina y se deja fijo en función de las necesidades del cliente.
- En ágil se va definiendo, refinando y adaptando en base a las retroalimentaciones del cliente.

Tiempo - ¿Cuánto tiempo me llevaría?

- Define la fecha de calendario en la cual se compromete a finalizar el proyecto y entregarle el producto resultante al cliente.
- En las metodologías ágiles, el tiempo es fijo.

Costo - ¿Cuánto debería costar?

- Define el costo de todos los recursos implicados en el desarrollo del proyecto. Esto incluye equipamiento, infraestructura, equipos, salarios, entre otros.

El costo y el tiempo del proyecto varían de forma directa con la definición del alcance del producto, es decir, el alcance es directamente proporcional al tiempo y al costo. Cuando el alcance aumenta (mayor cantidad de funcionalidades), el tiempo y dinero (costos) necesarios también deben aumentar, para lograr abordar un proyecto más grande.

Líder de Proyecto (no existe en ágil)

- En un enfoque tradicional, el líder de proyecto realiza la toma de decisiones en su totalidad sin la inclusión de la opinión del equipo.
- El líder asigna las tareas que deben realizar los integrantes del equipo, y además, maneja todas las relaciones con todos los stakeholders del proyecto (clientes, gerencias, contratistas, stakeholders en general).
- Trabaja con 3 variables que se conoce como la triple restricción.
- Aptitudes:
 - Motivación: habilidad para alentar al personal técnico a producir a su máxima capacidad.
 - Organización: habilidad para adaptar el proceso existente (o inventar nuevos), para lograr que el concepto inicial se transforme en el producto final.
 - Innovación: habilidad para alentar a las personas a crear y sentirse creativas.
 - Empatía.
 - Comunicación.
 - Liderazgo.
- También cuenta con habilidades duras, relacionadas a los conocimientos del producto, técnicas y herramientas.

- Resolución de problemas: identifica conflictos técnicos y organizativos, estructura la solución o motiva a otros profesionales para que la desarrollen, aplica lecciones aprendidas en otros proyectos y adapta el proceso de resolución ante inconvenientes.
- Identidad administrativa: tiene la confianza de asumir el control cuando es necesario.
- Logro: recompensa las iniciativas y los logros, incentivando al equipo a correr los riesgos de manera controlada.
- Influencia: define la estructura del equipo en función de la retroalimentación que él mismo suministra mediante palabras y gestos.
- Entre las responsabilidades del líder de proyecto se encuentran, en las nuevas metodologías:
 - Definir el alcance del proyecto.
 - Identificar a los involucrados, recursos y presupuesto.
 - Detallar las tareas, estimar los tiempos y requerimientos.
 - Identificar y evaluar riesgos.
 - Preparar planes de contingencia.
 - Controlar hitos y participar en las revisiones de las fases del proyecto.
 - Administrar el proceso de control de cambios.
 - Producir reportes de estado.

Equipo de proyecto

- Es un grupo de personas comprometidas con alcanzar un conjunto de objetivos de los cuales se sienten mutuamente responsables.
- Características:
 - Cuentan con conocimientos, habilidades técnicas, motivación, experiencias y diversas personalidades. Además, deben tener un sentimiento de mejora continua y aprendizaje mutuo.
 - Usualmente son un grupo pequeño, esto permite lograr una comunicación efectiva entre los miembros del equipo.
 - Trabajan en forma conjunta con espíritu de grupo, convirtiéndose en un equipo cohesivo donde prima la sinergia de este. (el equipo debe ser estable)
 - Sentido de responsabilidad como una unidad y se focalizan en el cumplimiento de las metas grupales.

Stakeholders

Son los interesados del proyecto, incluye el equipo de proyecto, el equipo de dirección, el líder de proyecto y el patrocinador.

Plan de proyecto (plan de desarrollo de software en el ámbito del software) → Es en tradicionales, en ágil tenemos el producto backlog, el sprint backlog, plan de release

- Es un artefacto de gestión que cumple la función de “hoja de ruta” de un proyecto, en la cual se documentan todas las decisiones que se toman a lo largo del desarrollo; en términos de: el proceso, la división de tareas a realizar, asignación de responsabilidades, equipos de trabajo, calendario de desarrollo, actividades de soporte.
- En el se establece qué se va a hacer (alcance del proyecto), cuándo se va a hacer (calendario), cómo se va a hacer (actividades o tareas para cubrir el alcance) y quién lo va a hacer.
- Implica las siguientes actividades:

- Definición del alcance del proyecto.
- Definición de proceso y ciclo de vida.
- Estimación.
- Gestión de riesgos.
- Asignación de recursos.
- Programación de proyectos.
- Definición de métricas.
- Definición de controles.

Definición del alcance del proyecto

- El alcance de un proyecto, junto a su objetivo, responden al qué se está desarrollando. Es importante diferenciar el alcance de un producto y el del proyecto en sí.
- El alcance de un producto define qué funcionalidades debe realizar el software para satisfacer los requerimientos del cliente y se mide contra la Especificación de requerimientos.
- El alcance del proyecto define todo el trabajo y solo el trabajo necesario que se debe realizar para cumplir con el desarrollo de este y poder entregar el producto o servicio con todas las características y funciones especificadas. Se mide contra el Plan de Proyecto (objetivos del proyecto).
- Tanto el alcance como el objetivo de un proyecto puede ser modificado a lo largo del proyecto, pero esto no necesariamente tiene que cambiar el alcance del producto. Pero, el alcance de un producto sí condiciona los alcances de un proyecto. (no siempre)
- Es importante aclarar que el alcance del proyecto es menor que el alcance del producto.
 - realizar la toma de requerimientos.
 - realizar Testing de determinados componentes
 - hacer el desarrollo del código de un componente

Ej de alcances de un proyecto

Definición del proceso y ciclo de vida

- Al definir el proceso que se va a utilizar, se responde al cómo se va a desarrollar el proyecto.
- Es importante tener en cuenta la información del contexto del desarrollo: objetivos, alcances, personas y recursos disponibles, clientes, forma de trabajo de personas.
- También se debe tener en cuenta la complejidad del software a desarrollar (alcances y objetivos) para así poder adaptar el proceso definido que se elija al proyecto en el cual se está trabajando.
- El ciclo de vida del proyecto es lo que define cuánto de cada tarea se debe hacer y en qué momento del desarrollo del proyecto hacerlo. Define:
 - Qué trabajo técnico debería realizarse en cada fase.
 - Quién debería estar involucrado en cada fase.
 - Cómo controlar y aprobar cada fase.
 - Cómo deben generarse los entregables.
 - Cómo revisar, verificar y validar el producto.
- En metodologías tradicionales, se permite elegir cualquier tipo de ciclo de vida (cascada, iterativo e incremental, espiral, etc.), en cambio en metodologías ágiles esto no es posible (si o si tiene que ser iterativo e incremental).

Estimación (Definido más arriba)

En tradicional estimamos 4 cosas: alcance/tamaño, esfuerzo (horas lineales), tiempo/cronograma, recursos/costos. Las estimaciones son absolutas (horas, días, semanas).

Aclaración: en scrum estimamos story points. Las estimaciones son por comparación

Gestión de riesgos

- Un riesgo es la probabilidad de ocurrencia de una pérdida o daño que afecte de forma directa al proyecto y pueda comprometer el incumplimiento de su objetivo.
- Es de suma importancia identificar y definir los riesgos a los que está atado un proyecto, para así poder analizarlos, y realizar una cuantificación de estos para así definir a cuáles se debe atender y sobre cuáles se debe hacer un seguimiento.
- Se clasifican en:
 - Riesgos del proyecto: aquellos que alteran el calendario o los recursos del proyecto. Un ejemplo de riesgo de proyecto es la renuncia de un diseñador experimentado.
 - Riesgos del producto: aquellos que afectan la calidad o el rendimiento del software a desarrollar. Un ejemplo de riesgo de producto es la falla que presenta un componente que se adquirió al no desempeñarse como se esperaba.
 - Riesgos organizacionales o de negocio: aquellos que afectan la calidad o el rendimiento del software a desarrollar. Un ejemplo de riesgo de producto es la falla que presenta un componente que se adquirió al no desempeñarse como se esperaba.
 - Riesgos organizacionales o de negocio: aquellos que afectan a la organización que desarrolla o que adquiere el software. Por ejemplo, un competidor que introduce un nuevo producto.
- Para medir los riesgos se utiliza lo que se conoce como **exposición al riesgo** (probabilidad x impacto) y como no se puede gestionar todos, se gestionan los que más exposición al riesgo tienen.
- Ante los riesgos identificados, es posible tomar 3 actitudes:
 - Negación: hacer de cuenta que los riesgos no existen.
 - Gestión Reactiva: esperar a que el riesgo ocurra y recién ahí ver que hacer.
 - Gestión Proactiva: ir trabajando sobre el riesgo generado planes de mitigación y de contingencia.
- Al tomar una actitud proactiva, la identificación y análisis de estos riesgos, permite definir planes de mitigación y contingencia, que permiten actuar en caso de que un riesgo suceda, y poder volver a la normalidad lo antes posible.
- Resumen de pasos:
 - Identificación de riesgos: se identifican los riesgos que suponen una mayor amenaza al proceso de ingeniería de software, al software a desarrollar o a la organización que lo desarrolla.
 - Análisis de riesgos: para cada riesgo identificado se realiza un juicio acerca de la probabilidad y del impacto.
 - Planeación del riesgo: para cada riesgo analizado, se definen estrategias para manejarlos. Estas estrategias consisten en considerar acciones a tomar para minimizar o evitar el impacto sobre el proyecto.
 - Monitorización del riesgo y control: Proceso que permite determinar que las suposiciones acerca de los riesgos de producto, proceso y organización no han cambiado.
 - Aprendizaje:

- A partir del aprendizaje trabajamos o no sobre una nueva lista de riesgos.

Asignación de responsabilidades (o recursos)

- La asignación de responsabilidades a personas, permite asignar roles a los integrantes del equipo de trabajo. En el entorno de procesos definidos (PUD, por ejemplo), estos roles se ven definidos en el concepto de trabajadores.
- Dentro de un mismo proyecto, una persona puede desenvolverse en más de un rol (por falta de presupuesto o porque el proyecto es chico)
- Para dejarlos explícitos se construye una tabla, la cual contiene la información de la persona, el rol y responsabilidades de esta en el contexto del proyecto.
- Se debe hacer mucho foco en la selección del personal de un proyecto, ya que las capacidades de los integrantes del equipo afectan a la calidad del software y al correcto desarrollo del proyecto en términos de tiempo.

Programación de proyectos (calendarización)

- En esta etapa se trata de definir en detalle, cada tarea que debe ser cumplida en el desarrollo.
- Se toma como base lo definido en la estimación del calendario realizada previamente, pero se refina al máximo detalle posible cada actividad.
- Cuando se habla de detallar las tareas, se hace referencia a descomponer el proceso de desarrollo en actividades refinadas a nivel de días, identificando quien la realiza, cuando debe realizarse y cuánto esfuerzo debe llevar en forma teórica.
- Generalmente la calendarización se realiza a través de un Diagrama de Gantt, realizado a través de alguna herramienta, como Microsoft Project.

Definición de métricas

- Las métricas se definen como una medida numérica que aporta visibilidad sobre el avance o estado del proyecto, proceso o producto en un momento determinado del desarrollo.
- Al realizar la planificación del proyecto, es necesario definir de forma clara y no ambigua, cada una de las métricas asociadas a cada dominio en conjunto con la forma de calcularlas.
- es imprescindible que las métricas sean representativas para el entorno, es decir, que representen un aumento en la información y beneficien la practicidad del desarrollo y seguimiento.
- Las métricas del proyecto se consolidan para crear métricas de proceso que sean públicas para toda la organización del software.
- A diferencia de las metodologías ágiles, acá se definen métricas que miden y exponen los avances. En cambio en ágil, el mayor indicador de avance de un producto, son los incrementos que se entregan en cada iteración. Las métricas básicas para un proyecto de software se clasifican en: Tamaño del producto, Esfuerzo, Calendario (tiempo), Defectos, Costos.



Definición de controles

- En la planificación de los controles del desarrollo, se toma como base las métricas ya definidas, y verifica que todo se está haciendo en concordancia con lo establecido. En esta planificación se definen reuniones periódicas, reportes o informes necesarios.

Planes de Soporte

- Planes de Testing.
- Planes de Mantenimiento.
- Planes de subcontrataciones.
- Planes de calidad.
- Planes de capacitaciones.
- Planes de iteraciones (si el ciclo de vida es iterativo/incremental).

Causas de fracasos de proyectos

- Fallas de toma de REQUERIMIENTOS (el 80%).
- Fallas al definir el problema.
- Planificar basado en datos insuficientes.
- La planificación la hizo el grupo de planificaciones.
- No hay seguimiento del plan de proyecto.
- Plan de proyecto pobre en detalles.
- Planificación de recursos inadecuada.
- Las estimaciones se basaron en “supuestos” sin consultar datos históricos.
- Nadie estaba a cargo.

Vínculo proceso-proyecto-producto en la gestión de un proyecto de desarrollo de software.

- Se dice que el proceso, automatizado con herramientas, se adapta al proyecto, al cual se incorporan personas, y de este se obtiene como resultado un producto.
- En el enfoque tradicional, previo a la definición del proyecto, es necesario determinar los objetivos y el ámbito del producto para de esta manera poder realizar las estimaciones pertinentes.
- Para obtener como resultado un producto o servicio (ya sea SaaS o SaaS), es necesario que el proyecto adopte un proceso de desarrollo para poder definir qué actividades, métodos, prácticas y transformaciones se utilizarán, mediante las cuales se va a obtener el software.

- Dentro de cada proyecto de desarrollo de software, es necesario utilizar un proceso diferente dependiendo del tipo de producto que se desea desarrollar, ya que puede que la complejidad de este sea un determinante en la elección del proceso. Por esto, es necesario adaptar los procesos a cada uno de los proyectos en los cuales se trabaja.
- El **proceso** proporciona un marco conceptual en donde se establece un plan completo para el desarrollo del software. Define como se va a desarrollar el software, estableciendo un conjunto de actividades.
- Las disciplinas de soporte son independientes del marco conceptual y recubren al modelo del proceso, es decir que atraviesan todas las actividades del proceso de desarrollo. Se debe definir el modelo de proceso a utilizar: secuencial, iterativo o recursivo. Luego, el marco conceptual que incluye las actividades fundamentales del desarrollo del software se adapta al modelo elegido.
- Un **proyecto** de desarrollo está integrado por un equipo de trabajo, dentro del cual deben estar los roles bien definidos, para que cada uno de los integrantes asuma la responsabilidad que le corresponda, y se tengan en claro los alcances de cada uno de estos roles.
- Las **personas** son la parte más importante, debido a que el desarrollo de software es una actividad humano-intensiva.



SCRUM

- Es un marco ligero que ayuda a las personas, equipos y organizaciones a generar valor a través de soluciones adaptables para problemas complejos.
- En pocas palabras, Scrum requiere un Scrum Master para fomentar un entorno donde:
 - Un propietario del producto (Product Owner) ordena el trabajo de un problema complejo en un Product Backlog.
 - El equipo de Scrum convierte una selección del trabajo en un Incremento de valor durante un Sprint.
 - El equipo de Scrum y sus partes interesadas (stakeholders) inspeccionan los resultados y realizan los ajustes necesarios para el próximo Sprint.
 - Repetir.
- El marco de Scrum es deliberadamente incompleto, solo define las partes necesarias para implementar la teoría de Scrum.
 - Scrum se basa en la inteligencia colectiva de las personas que lo utilizan.
 - En lugar de proporcionar a las personas instrucciones detalladas, las reglas de Scrum guían sus relaciones e interacciones.
- Scrum se basa en el empirismo y el pensamiento Lean.
 - El empirismo afirma que el conocimiento proviene de la experiencia y la toma de decisiones basadas en lo que se observa.
 - El pensamiento Lean reduce los desperdicios y se centra en lo esencial.

- Emplea un enfoque iterativo e incremental para optimizar la previsibilidad y controlar el riesgo.
- Scrum involucra a grupos de personas que colectivamente tienen todas las habilidades y experiencia para hacer el trabajo y compartir o adquirir tales habilidades según sea necesario.
- Scrum combina cuatro eventos formales para la inspección y adaptación dentro de un evento contenedor, el Sprint.
 - Estos eventos funcionan porque implementan los pilares empíricos de Scrum

Pilares de Scrum

- Transparencia:
 - El proceso y el trabajo emergentes deben ser visibles para aquellos que realizan el trabajo, así como para los que reciben el trabajo.
 - Con Scrum, las decisiones importantes se basan en el estado percibido de sus tres artefactos formales.
 - Los artefactos que tienen poca transparencia pueden conducir a decisiones que disminuyen el valor y aumentan el riesgo. La transparencia permite la inspección.
 - La inspección sin transparencia genera engaños y desperdicios.
- Inspección:
 - Los artefactos de Scrum y el progreso hacia objetivos acordados deben ser inspeccionados con frecuencia y diligentemente para detectar varianzas o problemas potencialmente indeseables.
 - Para ayudar con la inspección, Scrum proporciona cadencia en forma de sus cinco eventos. Los eventos de Scrum están diseñados para provocar cambios.
 - La inspección permite la adaptación. La inspección sin adaptación se considera inútil.
- Adaptación:
 - Si algún aspecto de un proceso se desvía fuera de los límites aceptables o si el producto resultante es inaceptable, el proceso que se está aplicando o los materiales que se producen deben ajustarse.
 - El ajuste debe realizarse lo antes posible para minimizar la desviación adicional.
 - La adaptación se vuelve más difícil cuando las personas involucradas no están empoderadas o no poseen capacidad para autogestionarse. Se espera que un equipo de Scrum se adapte en el momento en que aprenda algo nuevo por medio de la inspección.

Valores de Scrum

- El uso exitoso de Scrum depende de que las personas sean más competentes en vivir cinco valores: **Compromiso, Enfoque, Apertura, Respeto y Coraje**.
- Los miembros del equipo de Scrum se respetan mutuamente para ser personas capaces e independientes, y son respetados como tales por las personas con las que trabajan.
- Los miembros del equipo de Scrum tienen el valor de hacer lo correcto y de trabajar en problemas complejos.
- Estos valores dan dirección al equipo de Scrum con respecto a su trabajo, acciones y comportamiento.

- Cuando estos valores son asimilados por el equipo de Scrum y las personas con las que trabajan, los pilares empíricos de Scrum cobran vida construyendo confianza.

El equipo Scrum (Scrum Team)

- La unidad fundamental de Scrum es un pequeño equipo de personas.
- consta de un Scrum Master, un propietario de producto (Product Owner) y desarrolladores.
- No hay subequipos ni jerarquías.
- son multifuncionales, lo que significa que los miembros tienen todas las habilidades necesarias para crear valor en cada Sprint.
- También son autogestionados, lo que significa que internamente deciden quién hace qué, cuándo y cómo.
- El equipo de Scrum es lo suficientemente pequeño como para permanecer ágil y lo suficientemente grande como para completar un trabajo significativo dentro de un Sprint (10 o menos personas)
- Si los equipos de Scrum se vuelven demasiado grandes, se debe considerar la reorganizarse en varios equipos cohesionados, cada uno centrado en el mismo producto, compartiendo el mismo objetivo de producto, trabajo pendiente del producto (Product Backlog) y propietario del producto (Product Owner).
- Trabajar en Sprints a un ritmo sostenible mejora el enfoque y la consistencia del equipo de Scrum.
- Todo el equipo de Scrum es responsable de crear un incremento valioso y útil en cada Sprint.
- **Scrum define tres responsabilidades específicas dentro del equipo de Scrum: los desarrolladores, el propietario del producto (Product Owner) y el Scrum Master.**

Desarrolladores

- Son las personas del equipo Scrum que se comprometen a crear cualquier aspecto de un Incremento útil (funcional) en cada Sprint.
- Sus habilidades son amplias y variarán con el dominio del trabajo.
- Sin embargo, los desarrolladores siempre son responsables de:
 - Crear un plan para el Sprint, el Sprint Backlog;
 - Inculcar la calidad adhiriéndose a una definición de Hecho;
 - Adaptar su plan cada día hacia el Objetivo Sprint;
 - Responsabilizarse mutuamente como profesionales.

Propietario del producto (Product Owner)

- Es responsable de maximizar el valor del producto resultante del trabajo del equipo de Scrum. La forma en que hace esto puede variar ampliamente entre organizaciones, equipos Scrum e individuos.
- El Propietario del Producto también es responsable de la gestión eficaz de la pila del producto (Product Backlog), que incluye:
 - Desarrollar y comunicar explícitamente el Objetivo del Producto;
 - Creación y comunicación clara de elementos de trabajo pendiente del producto;
 - Pedido de artículos de trabajo pendiente del producto;
 - Asegurarse de que el trabajo pendiente del producto sea transparente, visible y comprendido.

- Puede hacer todo lo anterior o puede delegarlo a otros. En cualquier caso, el propietario del producto sigue siendo responsable.
- Es solo una persona y para que tengan éxito, toda la organización debe respetar sus decisiones.

Scrum Master

- Es responsable de establecer Scrum tal como se define en la Guía de Scrum. Ayudando a todos a comprender la teoría y la práctica de Scrum, tanto dentro del Equipo como en toda la organización.
- Es responsable de la efectividad del Scrum Team.
- Sirve al equipo de Scrum de varias maneras, incluyendo:
 - Capacitar a los miembros del equipo en autogestión y multifuncionalidad;
 - Ayudar al equipo de Scrum a centrarse en la creación de incrementos de alto valor que cumplan con la definición de hecho;
 - Promover la eliminación de los impedimentos para el progreso del equipo Scrum;
 - Asegurar de que todos los eventos de Scrum se lleven a cabo, sean positivos, productivos y que se respete el tiempo establecido (time-box) para cada uno de ellos.
- Sirve al PO de varias maneras, incluyendo:
 - Ayudar a encontrar técnicas para una definición eficaz de los objetivos del producto y la gestión de los retrasos en el producto;
 - Ayudar al equipo de Scrum a comprender la necesidad de elementos de trabajo pendiente de productos claros y concisos;
 - Ayudar a establecer la planificación empírica de productos para un entorno complejo;
 - Facilitar la colaboración de las partes interesadas según sea solicitado o necesario.
- Sirve a la organización de varias maneras, incluyendo:
 - Liderar, capacitar y mentorizar a la organización en su adopción de Scrum;
 - Planificar y asesorar sobre la implementación de Scrum dentro de la organización;
 - Ayudar a las personas y a las partes interesadas a comprender y promulgar un enfoque empírico para el trabajo complejo;
 - Eliminar las barreras entre las partes interesadas y los equipos de Scrum.

Eventos/ceremonias de Scrum

- Cada evento en Scrum es una oportunidad formal para inspeccionar y adaptar los artefactos de Scrum.
- Están diseñados específicamente para permitir la transparencia necesaria.
- Si no se realizan los eventos según lo prescrito, se pierden oportunidades para inspeccionar y adaptarse.
- De manera óptima, todos los eventos se llevan a cabo al mismo tiempo y lugar para reducir la complejidad.

El Sprint → son el latido del corazón de Scrum, donde las ideas se convierten en valor.

- Son eventos de longitud fija de un mes o menos para crear consistencia.
- Un nuevo Sprint comienza inmediatamente después de la conclusión del Sprint anterior.
- Todo el trabajo necesario para alcanzar el objetivo del producto, como Sprint Planning, DailyS, Sprint Review y Sprint Retrospective ocurren dentro del Sprints.
- Durante el Sprint:
 - No se hacen cambios que pongan en peligro el Objetivo Sprint;
 - La calidad no disminuye;
 - El trabajo pendiente del producto se refina según sea necesario;
 - El alcance se puede clarificar y renegociar con el Propietario del Producto a medida que se aprende más.

- Cuando el horizonte de un Sprint es demasiado largo, el Objetivo de Sprint puede volverse obsoleto, la complejidad puede aumentar y el riesgo puede aumentar.
- Los Sprints más cortos se pueden emplear para generar más ciclos de aprendizaje y limitar el riesgo de coste y esfuerzo a un período de tiempo más pequeño.
- Cada Sprint puede considerarse un proyecto corto.
- Existen varias prácticas para pronosticar el progreso, como gráficos de burn-downs, burn-ups, o flujos acumulativos.
- Un Sprint podría ser cancelado si el Objetivo del Sprint se vuelve obsoleto. Solo el Propietario del Producto tiene la autoridad para cancelar el Sprint.

Planificación de Sprint (Sprint Planning)

- Inicia el Sprint estableciendo el trabajo que se realizará para el mismo.
- Este plan resultante es creado por el trabajo colaborativo de todo el equipo de Scrum.
- El propietario del producto (Product Owner) se asegura de que los asistentes estén preparados para discutir los elementos de trabajo pendiente de producto más importantes y cómo se asignan al objetivo del producto. El equipo de Scrum también puede invitar a otras personas a asistir para proporcionar asesoramiento
- Aborda los siguientes temas:
 - Tema Uno: ¿Por qué este Sprint es valioso?: El PO propone cómo el producto podría aumentar su valor y utilidad en el Sprint actual. Luego, todo el equipo colabora para definir un objetivo de Sprint que comunique por qué el Sprint es valioso para las partes interesadas.
 - Tema dos: ¿Qué se puede hacer este Sprint?: A través del debate con el PO, los desarrolladores seleccionan los elementos del Product Backlog para incluir en el Sprint actual. El equipo de Scrum puede refinar estos elementos durante este proceso, lo que aumenta la comprensión y confianza.
 - Tema Tres: ¿Cómo se realizará el trabajo elegido?: Para cada Product Backlog item seleccionado, los desarrolladores planifican el trabajo necesario para crear un incremento que cumpla con la definición de hecho.
- El objetivo de Sprint (Sprint Goal), los elementos de trabajo pendiente de producto seleccionados para el Sprint, más el plan para entregarlos se conocen conjuntamente como el trabajo pendiente de Sprint (Sprint Backlog).
- Duración máxima de ocho horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto.

Scrum Diario (Daily Scrum)

- Su propósito es inspeccionar el progreso hacia el Objetivo Sprint y adaptar el Sprint Backlog según sea necesario, ajustando el próximo trabajo planeado.
- es un evento de 15 minutos (máximo) para los desarrolladores del equipo de Scrum. Si el PO o el Scrum Master están trabajando activamente en los elementos del Trabajo pendiente de Sprint, participan como desarrolladores.
- Mejoran la comunicación, identifican impedimentos, promueven una rápida para la toma de decisiones, y en consecuencia, eliminan la necesidad de otras reuniones.

Revisión del Sprint (Sprint Review)

- El propósito es inspeccionar el resultado del Sprint y determinar futuras adaptaciones.

- El equipo de Scrum presenta los resultados de su trabajo a las partes interesadas clave y se discute el progreso hacia el Objetivo de Producto.
- Durante el evento, el equipo de Scrum y las partes interesadas revisan lo que se logró en el Sprint y lo que ha cambiado en su entorno. En base a esta información, los asistentes colaboran en qué hacer a continuación.
- Es una sesión de trabajo y el equipo de Scrum debe evitar limitarla a que se convierta en una simple presentación.
- Es el penúltimo evento del Sprint y se utiliza en un plazo máximo de cuatro horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto.

La retrospectiva del Sprint (Sprint Retrospective)

- Su propósito es planificar formas de aumentar la calidad y la eficacia.
- El equipo de Scrum inspecciona cómo fue el último Sprint con respecto a individuos, interacciones, procesos, herramientas y su definición de Hecho. Los elementos inspeccionados a menudo varían según el dominio del trabajo.
- El equipo de Scrum analiza qué fue bien durante el Sprint, qué problemas encontró y cómo esos problemas fueron (o no) resueltos.
- El equipo de Scrum identifica los cambios más útiles para mejorar su eficacia.
- La retrospectiva Sprint concluye el Sprint.
- Se utiliza un intervalo de tiempo de hasta un máximo de tres horas para un Sprint de un mes. Para sprints más cortos, el evento suele ser más corto.

Refinamiento del Product Backlog

Artefactos de Scrum

- Representan trabajo o valor.
- Cada **artefacto contiene un compromiso** para garantizar que proporciona información que mejora la transparencia y el enfoque con el que se puede medir el progreso:
 - Para el trabajo pendiente del producto (producto backlog) es el objetivo del producto.
 - Para el Sprint Backlog es el Sprint Goal.
 - Para el Incremento es la Definición de Hecho.

Pila del producto (Product Backlog)

- El trabajo pendiente del producto es una lista emergente y ordenada de lo que se necesita para mejorar el producto.
- Es la única fuente de trabajo emprendida por el equipo Scrum.
- Los elementos de trabajo pendiente de producto que puede ser hecho por el equipo de Scrum dentro de un Sprint se consideran listos para su selección en un evento de planificación de Sprint.
- El refinamiento de Backlog del producto es el acto de descomponer y definir aún más los elementos de trabajo pendiente del producto en artículos más pequeños y precisos. Esta es una actividad en curso para agregar detalles, como una descripción, un pedido y un tamaño. Los atributos a menudo varían con el dominio del trabajo.
- Los desarrolladores que realizarán el trabajo son responsables del tamaño. El PO puede influir en los desarrolladores ayudándoles a entender y seleccionar mejores alternativas.
- Compromiso: Objetivo del producto (Product Goal)

- Describe un estado futuro del producto que puede servir como objetivo para el equipo Scrum contra el cual planificar.
- El objetivo del producto se encuentra en el trabajo pendiente del producto (Product Backlog).
- El resto del trabajo pendiente del producto surge para definir "qué" cumplirá el objetivo del producto.
- El objetivo del producto es el objetivo a largo plazo para el equipo Scrum. Deben cumplir (o abandonar) un objetivo antes de asumir el siguiente.

La pila del Sprint (Sprint Backlog)

- El Trabajo pendiente de Sprint se compone del objetivo sprint (por qué), el conjunto de elementos de trabajo pendiente de producto seleccionados para el Sprint (qué), así como un plan accionable para entregar el incremento (cómo).
- El Trabajo pendiente de Sprint es un plan por y para los desarrolladores. Es una imagen muy visible y en tiempo real del trabajo que los desarrolladores planean realizar durante el Sprint para lograr el Objetivo Sprint. Por lo tanto, el Sprint Backlog se actualiza a lo largo del Sprint a medida que se aprende más. Debe tener suficientes detalles para que puedan inspeccionar su progreso en el Scrum Diario.
- Compromiso: Sprint Goal
 - El Sprint Goal es el único objetivo para el Sprint.
 - Aunque el objetivo de Sprint es un compromiso de los desarrolladores, proporciona flexibilidad en términos del trabajo exacto necesario para lograrlo. Crea coherencia y enfoque, animando al equipo de Scrum a trabajar juntos en lugar de en iniciativas separadas.
 - El objetivo de Sprint se crea durante el evento Sprint Planning y, a continuación, se agrega al Trabajo pendiente de Sprint.

Incremento (Increment)

- Un Incremento es un paso hacia el Objetivo del Producto.
- Cada Incremento es aditivo a todos los Incrementos anteriores y verificado a fondo, asegurando que todos los Incrementos funcionen juntos.
- Para proporcionar el valor, el incremento debe ser utilizable.
- Se pueden crear varios incrementos dentro de un Sprint. La suma de los Incrementos se presenta en la Revisión Sprint apoyando así el empirismo.
- Compromiso: Definición de Hecho (Definition of Done)
 - La Definición de Hecho es una descripción formal del estado del Incremento cuando cumple con las medidas de calidad requeridas para el producto.
 - En el momento en que un elemento de trabajo pendiente de producto cumple con la definición de hecho, se crea un incremento.
 - Si un elemento de trabajo pendiente de producto no cumple con la definición de hecho, no se puede liberar, ni siquiera presentar en la revisión de Sprint. En su lugar, vuelve al Trabajo pendiente del producto para su consideración futura.

Timebox

- Existe para que no se pierda más tiempo del necesario en las distintas tareas y para aprender a negociar en términos del alcance del producto y no del tiempo o los costos.
- Si no se llega al final de un sprint, en vez de extender el tiempo, se entrega menos.
- Todas las ceremonias son Timebox. El refinamiento del Product Backlog al ser una actividad continua, no tiene definido un tiempo exacto, sino que el tiempo se expresa en términos porcentuales sobre la definición del Sprint.

Sprint: 1 mes o menos
Sprint Planning: 8 horas máximo para un Sprint de 1 mes
Daily Meeting: 15 minutos
Sprint Review: 4 horas máximo para un Sprint de 1 mes
Sprint Retrospective: 3 horas máximo para un Sprint de 1 mes
Refinamiento del PB: 10% del tiempo de Sprint

Definición de Listo (DoR) y Definición de hecho (DoD)

- El DoR es un criterio que define cuando una US está lo suficientemente bien formulada como para poderse incluir en un Sprint. Por ejemplo, que cierta US debe tener un prototipo, entonces cuando el prototipo esté realizado, el DoR dirá “se ha definido el prototipo para la US”.
 - La base del DoR es el modelo invest.
- El DoD es un criterio que dice cuando una historia está lo suficientemente bien implementada como para entrar al Sprint Review y ser mostrada al PO, entonces él será el encargado de aprobar o no la US y en caso de que esté aprobada decidirá si desea ponerla en producción. En el DoD se arma un checklist con todo lo que debe cumplir una US para entrar en producción.
 - El DoD depende del equipo, no hay un estándar.

Definición de "Listo" (Ready)
☐ Valor de negocio claramente expresado.
☐ Detalles suficientemente comprendidos por el Equipo de forma tal que puedan tomar una decisión informada sobre si pueden completar el ítem del product Backlog (PBI).
☐ Dependencias identificadas y no hay dependencias externas que puedan impedir que el PBI se complete.
☐ El equipo ha sido asignado adecuadamente para completar el PBI.
☐ El PBI ha sido debidamente estimado y es lo suficientemente pequeño para ser completado en un Sprint.
☐ Los criterios de aceptación son claros y testeables.
☐ Los criterios de performance si hay, son claros y testeables.
☐ El equipo comprende como mostrar el PBI en la Sprint Review.

Métricas que se plantean en SCRUM → Se plantean 3 pero principalmente se usan 2

Definición de Hecho (DONE)
☐ Diseño revisado
☐ Código Completo
☐ Código refactorizado
☐ Código con formato estándar
☐ Código Comentado
☐ Código en el repositorio
☐ Código inspeccionado
☐ Documentación de Usuario actualizada
☐ Probado
☐ Prueba de unidad hecha
☐ Prueba de integración hecha
☐ Prueba de sistema hecha
☐ Cero defectos conocidos
☐ Prueba de Aceptación realizada
☐ En los servidores de producción

- La medición es una salida no una actividad.
- “Medir la que sea necesario y nada mas”. Sabiendo que la principal medida de progreso es el software funcionando.

Capacidad del equipo en un sprint

- Es una métrica que no se toma (mide), si no que se estima.
- Sirve para determinar cuánta cantidad de trabajo puede comprometer un equipo para un determinado Sprint.
- Se hace en el Sprint Planning y teniendo en cuenta la capacidad, se contrasta en cuántas US del Product Backlog se pueden incorporar en el Sprint Backlog.
- Para equipos más maduros, la capacidad puede ser estimada en Story Points (medida de producto) y para equipos menos maduros se puede estimar en horas ideales (medida de trabajo).
 - Las story points si se consumieron los puntos de una US, quiere decir que ya esta terminada. Pero si es en cuestión de horas es mas difícil saber si esa US significa que se termino de realizar.

Velocidad

- Es una métrica del progreso de trabajo de un equipo. Se calcula (no se estima) sumando el número de Story Points asignados al estimar una US, que el equipo completa durante cada iteración, lo cual implica que la US se ha desarrollado y testeado por completo (DoD), y además el Product Owner la ha aceptado.

- Si la US se completó parcialmente, no se cuentan sus Story Points en esa iteración.
- Esto implica que el equipo no se enfoque en la velocidad de cada iteración, sino más bien, en el promedio de velocidades a lo largo de sucesivas iteraciones.
 - Esto brindará una mayor certeza acerca de la medida de progreso de trabajo del equipo.
- Esta métrica permite retroalimentar el proceso de estimación, un buen equipo puede permitirse estimar cuántos puntos de historia puede desarrollar en la siguiente iteración a través del cálculo de velocidad.

Running teste features (RTF) → es la que menos se usa

- Cuenta la cantidad de features que ya están funcionando.
- Pero no es representativo porque hay features chicas y otras grandes. Tener mucho no significa tener la mayoría.

Métricas en ambientes tradicionales

Métricas de proceso:

- Son métricas estratégicas a nivel organizacional, orientadas a mejorar el proceso y tienen como objetivo generar indicadores que permitan mejorar los procesos de software a largo plazo. Son públicas y se despersonifican las mediciones de personas, áreas o proyectos particulares, para obtener un número aislado y tener una métrica sobre el proceso de la organización en general, como un todo.
- Son responsabilidad del Ingeniero de Procesos, quien realiza las mediciones y luego publica los datos para que todos los empleados de la organización puedan acceder a ellos.
- Los indicadores de proceso permiten tener una visión profunda de la eficacia de un proceso, determinando qué funciona de acuerdo a lo esperado y qué no.
- Proporcionan beneficios significativos a medida que la organización trabaja para mejorar su nivel global de madurez del proceso.

Métricas de proyecto:

- Están enfocadas a los recursos que se dedican al proyecto, como costos, esfuerzos, estimaciones y tiempo. Son responsabilidad del Líder de Proyecto y permiten al equipo adaptar el desarrollo de los proyectos y de las actividades técnicas. Estas métricas son privadas de ese proyecto y solo son visibles para los involucrados en el mismo.
- Se utilizan para mejorar la planificación del desarrollo, generando ajustes que eviten retrasos, reduzcan riesgos potenciales y por lo tanto, problemas.
- Además, se utilizan para evaluar la calidad de los productos en todo momento y en caso de ser necesario, modificar el enfoque para mejorar la calidad, minimizando defectos, retrabajo y por ende el costo total del proyecto.
- Las métricas de proyecto se consolidan con el fin de crear métricas de procesos que sean públicas para la organización de software como un todo.

Métricas de Producto:

- Están enfocadas en lo que se construye, son responsabilidad del equipo de desarrollo y de Testing y son particulares de ese producto.

- Se utilizan con propósitos técnicos y tienen como objetivo generar indicadores en tiempo real de la eficacia del análisis, el diseño, la estructura del código, la efectividad de los casos de prueba y calidad del software a construir.
- Se deben controlar los artefactos resultantes del proceso de desarrollo (componentes y modelos) para garantizar:
 - Que cumplan con los requerimientos del cliente;
 - Que cumplan con los requerimientos de calidad;
 - Que estén libre de errores;
 - Que se realizaron bajo los procedimientos de calidad.

Ambiente de trabajo

El equipo define sus horarios, pizarrones/postit, abierto (el silencio es un mal signo)

Herramientas de SCRUM

Taskboard - Tarjeta de tarea

Es una tarjeta que cuenta con 5 partes:

1. Código del Ítem Backlog (opcional): es un identificador del ítem en el Product Backlog.
2. Nombre de la actividad: suele ser una frase verbal que define qué es lo que hay que hacer.
3. Iniciales del responsable de realizar la tarea.
4. Número de día del sprint (opcional)
5. Estimación del tiempo para finalizar la tarea.



Story	To Do		In Process	To Verify	Done
As a user, I... 8 points	Code the... 9	Test the... 8	Code the... DC 4	Test the... SC 6	Code the... SC 8
	Code the... 2	Code the... 8	Test the... SC 8		Test the... SC 6
	Test the... 8	Test the... 4			Test the... SC 6
As a user, I... 5 points	Code the... 8	Test the... 8	Code the... DC 8		Test the... SC 6
	Code the... 4	Code the... 6			Test the... SC 6

Tablero utilizado por el Equipo de Desarrollo en el cual se colocan las tarjetas de tareas a realizar durante el sprint. Se encuentra dividido en 5 secciones:

- Story: el nombre de la historia (con estimación en story points).
- To Do: aquí se encuentran las tareas por hacer en el Sprint (con estimación en horas).
- In Process: aquí se encuentran las tareas que se están realizando, las tareas se toman (método pull).
- To Verify: aquí se encuentran las tareas finalizadas que deben verificarse.
- Done: aquí se encuentran las tareas terminadas: finalizadas y verificadas.
- Done done: se encuentran las US terminadas (es opcional).

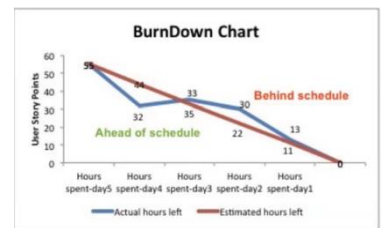
Lo ideal es que el nivel de granularidad de las tareas sea lo más fino posible, de tal forma que se detalle profundamente el avance del sprint, teniendo varias tareas en cada sección del tablero.

Gráficos del Backlog

Brindan información acerca del progreso de un Sprint, de una release o del producto.

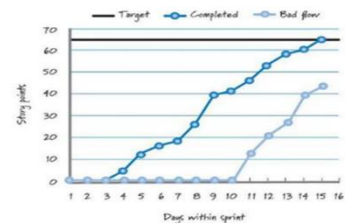
Sprint Burn-Down Chart

Visualiza con una curva descendente cuánto trabajo queda para terminar. En el eje de las abscisas se coloca el tiempo que va desde el inicio del sprint. En el eje de las ordenadas se coloca los puntos de historia a realizar en el sprint.



Sprint Burn-Up Chart

Visualiza con una curva ascendente cuánto trabajo se ha completado a lo largo de la release, es decir que visualiza los puntos de historia terminados en cada sprint.



Combined Burn Chart

Visualiza cuánto trabajo queda para terminar y cuanto trabajo se ha realizado.

Planificación (basado en lo que está en la página 31)

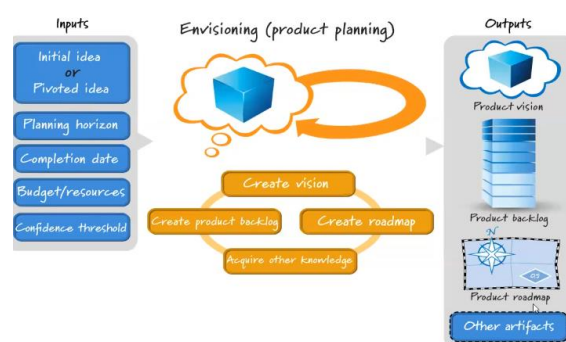
- En agile y en scrum se planifica, pero se planifica en distintos niveles.

Planificación de portfolio

Lo que esperamos tener como resultado es: Tengo esta visión de producto y quiero planificar como sería mi portfolio para saber si vale la pena invertir en mi visión.

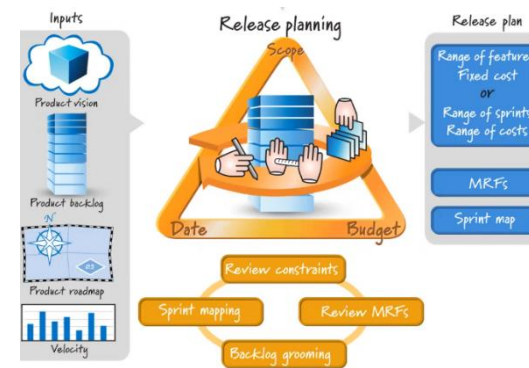
Planificación de producto

- Es un nivel menor al de portfolio.
- Sabemos que paso por el filtro estratégico, por el filtro de decisión económica.
- Ahora debemos definir cuantos reléase necesito para tener el producto que yo quiero.
- En resumen, tengo mi producto backlog que es dinámico (se van agregando y quitando cosas) y tengo mi roadmap. Con esto yo debería poder estimar cuantos reléase necesito para lograr el producto.
- Las restricciones son las mismas que la triple restricción en lo tradicional.



Planificación de Release

- Ahora, habiendo estimado cantos reléase voy a tener, tengo que estimar cuales van a ser las US que van a ser incluidas en ese reléase (recordando que el producto backlog puede variar).
- En base a esas US/Feature que quiero implementar, estimo cuantos sprint voy a necesitar para poder construir/implementar cada una de esas Feature. Por lo que haga una estimación en función de la capacidad del equipo para armar el plan de reléase.
- Se siguen teniendo en cuenta el alcance, el tiempo y el presupuesto



Planificación sprint

- En base al plan de reléase, voy trabajando con cada uno de los sprint concretos teniendo ya la lista de features y si es necesario puedo descomponerlo en tareas.

COSAS QUE AGREGUE

El producto tiene un ciclo de vida (idea, plan de negocio, producto, operaciones, retiro) y el alcance del producto son todas las características, features que el producto incluye.

Pero el ciclo de vida del proyecto es mucho mas chico que el del producto, ya que en la vida de un producto se ejecuten seguramente muchos proyectos. Y el alcance del proyecto tiene que ver con el trabajo necesario para entregar el producto.

Entonces decimos que el alcance del producto se mide contra los requerimientos, mientras que el alcance del proyecto se mide contra el plan del proyecto